Improved Range Searching And Range Emptiness Under FHE Using Copy-And-Recurse

Kushnir Eyal
eyal.kushnir@ibm.com

Shaul Hayim
hayim.shaul@ibm.com

April 3, 2025

Abstract

Range counting is the problem of preprocessing a set $P \subset \mathbb{R}^d$ of n points, such that given a query range γ we can efficiently compute $|P \cap \gamma|$. In the more general range searching problem the goal is to compute $f(P \cap \gamma)$, for some function f. It was already shown (Kushnir et al. PETS'24) how to efficiently answer a range searching query under FHE using a technique they called Copy-and-Recurse to traverse partition trees. In the *Range emptiness* problem the goal is to compute only whether $P \cap \gamma = \emptyset$. This was shown (in plaintext) to be done more efficiently. Range emptiness is interesting on its own and also used as a building block in other algorithms. In this paper we improve and extend the results of Kushnir et al. First, for range searching we reduce the overhead term to the optimal $O(n)$, so for example if the ranges are halfspaces in $\mathbb{R}^d$ bounded by hyperplanes then range searching can be done with a circuit of size $O(t \cdot n^{1-1/d+\varepsilon} + n)$, where t is the size of the sub-circuit that checks whether a point lies under a hyperplane.

Second, we introduce a variation of copy-and-recurse that we call leveled copy-and-recurse. With this variation we improve range searching in the 1-dimensional case as well as traversal of other trees (e.g., binary trees and B-trees). Third, we show how to answer range emptiness queries under FHE more efficiently than range counting.

We implemented our algorithms and show that our techniques for range emptiness yield a solution that is $\times 3.6$ faster than the previous results for a database of 2^{25} points.

1 Introduction

The *range searching* problem has received a lot of attention, e.g. [Mat91, SS11] in plaintext and [KMS24, CKK15] in the secure settings. In its simplest version, we are given a set of n points $P \subset \mathbb{R}^d$ that we preprocess such that given a volume (range) $\gamma \subset \mathbb{R}^d$ we can quickly count $|P \cap \gamma|$. In the secure settings, the data set P and the range γ are secret and cannot be shared. For example, P may be medical records belonging to a hospital. An independent physician may want to estimate a drug's effectiveness for a patient with profile ρ using conditional probability

$$\Pr[\text{recovered} \mid \text{received drug and has profile } \rho]. \tag{1}$$

This equals to

$$(1) = \frac{\Pr[\text{recovered and received drug and has profile } \rho]}{\Pr[\text{received drug and has profile } \rho]} = \frac{P \cap \gamma_2}{P \cap \gamma_1},$$

and can be computed with 2 range counting queries, where each point $p \in P$ represents a single medical record and γ_1 is a range filtering the points corresponding to records with profile ρ who got the drug and γ_2 is a range filtering the points corresponding to records with profile ρ who got the drug and recovered.

Plaintext algorithms such as in [Mat91, Mat92, SS11] compute $|P \cap \gamma|$ efficiently by grouping points in P together and testing for each group whether it is: (1) contained in γ and then all the points in the group are counted; (2) disjoint from γ and then the group is ignored; or (3) partially contained in γ. In the latter case a finer partition of the group is used in a recursive way.

This example deals with medical data which requires complying with regulations such as HIPPA [U.S96] to protect its privacy. An easy way to ensure the security of the data set and/or the query is

to use secure computation protocols. These protocols can be based on Fully Homomorphic Encryption (FHE) or on Beaver triples [Bea92], Oblivious RAM (ORAM) [TCSK16, CFLM20, MMRS15], or Searchable Symmetric Encryption (SSE) [DPP+16, DPP+18, FMET22, CGKO06, FJK+15].

Our algorithms are generic and require only the existence of addition and multiplication operations therefore they can be implemented with any of the above solutions but in this paper we focus on and present them with FHE. We first briefly describe and compare the different schemes and explain why we prefer FHE. **ORAM** is a method that lets party A keep a database at an untrusted party B such that A can access the database without revealing the content of the database or the access pattern to it. In Multi-Client ORAM [CFLM20, MMRS15] the database can be accessed by multiple clients A_1, A_2 without leaking information. A drawback of this technique is it requires many rounds of communication. This is sometimes discouraged in real life if the latency of the communication is significant. Other secure protocols such as those that are based on **Beaver triples** also require many rounds of communication and suffer from the same drawback. **SSE** is an encryption scheme where 2 ciphertexts can be compared such that the output of the comparison is given in plaintext. With SSE the access pattern of the algorithm is revealed which might lead to data leakage. See for example attacks on SSE-based range searching [LMP18, GLMP18, GJW19, MFET23].

With **FHE**, parties encrypt their inputs and an output is computed without decrypting them (see below more details). FHE is more CPU intensive but the cost of FHE-based solution decreases as new FHE schemes, hardware, and algorithms are developed. Today several systems for statistics such as in the example above are based on FHE and are already available (e.g. by CryptoLab [hea24], Duality [dua24], IBM [ibm24]). Figure 1 depicts a FHE-based system where an encrypted data set P and encrypted queries γ are uploaded to an untrusted server that computes $|P \cap \gamma|$ without decrypting.

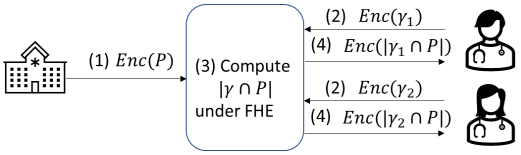

Figure 1: An untrusted system providing statistics-as-a-service using FHE. (1) The data owner (hospital) encrypts and uploads a data set. (2) A querier (doctor) encrypts their query and sends it to the untrusted cloud. (3) The cloud performs the query under FHE and (4) returns the result to the querier.

Answering a range searching query under FHE is less efficient than in plaintext even asymptotically. This is because FHE algorithms run in circuit model unlike plaintext algorithms that run in comparison model. This means that under FHE an algorithm cannot change its behavior depending on the input. Specifically, the algorithm cannot skip a group of points. Indeed, any range searching algorithm under FHE has a lower bound of $\Omega(n)$ as was already mentioned by [KMS24]. This is shown by solving the PIR problem using range searching and remembering that [BIM00] showed PIR has a lower bound of $\Omega(n)$. Intuitively, if a secure algorithm can answer a range searching query without accessing $p \in P$ then an adversary can conclude that $p \notin P \cap \gamma$, thus breaking the semantic security of FHE.

As noted by [KMS24] although there is a $\Omega(n)$ lower bound most of the running time is spent testing whether $p \in \gamma$, for multiple $p \in \mathbb{R}^d$ (not necessarily $p \in P$), and so we want to minimize the number of times this test is executed. They denoted the time (or more accurately the circuit size) to execute this test by t and showed how to efficiently convert a plaintext algorithm to run under FHE with the same time complexity and adding overhead that is linear in the size of the database. Their technique, which they named copy-and-recurse, considered an algorithm that partitions the problem into r sub-problems such that only $\xi < r$ of them need to be solved in recursion. To effectively solve only ξ sub-problems under FHE they made a copy of those ξ sub-problems and then recursed into the ξ copies. This copying is done using a selection matrix which can be thought of as a ξ-way MUX. Indeed, multiplying by the selection matrix adds overhead linear in the database size, but after paying that overhead the FHE algorithm has the same asymptotic time complexity as the plaintext.

Specifically, in the context of range searching they partitioned P into r subsets $P_1, \ldots, P_r$ using the partition theorems from [Mat91, AM94, AMS13] that guaranteed that for most subsets P_i it can be quickly verified whether $P_i \subset \gamma$ or $P_i \cap \gamma = \emptyset$ such that only $\xi = O(r^{1-1/d})$ subsets need to be recursed into.

	Ranges	[KMS24] (FHE)	This work (FHE)
	axis-parallel hyperboxes	$O(tn^{1+\varepsilon} + n\log^{d-1} n)$	$O(t\log^d n + n\log^{d-1} n)$
[Mat91]	hyperplanes	$O(tn^{1-\frac{1}{d}+\varepsilon} + n^{1+\varepsilon})$	$O(tn^{1-\frac{1}{d}+\varepsilon} + n)$
[AM94]	semialgebraic	$O(tn^{1-\frac{1}{c}+\varepsilon} + n^{1+\varepsilon})$	$O(tn^{1-\frac{1}{c}+\varepsilon} + n)$
[AMS13]	semialgebraic	$O(tn^{1-\frac{1}{d}+\varepsilon} + n^{1+\varepsilon})$	—
[Mat92]	hyperplanes (emptiness)	—	$O(tn^{1-\frac{1}{\lfloor d/2 \rfloor}+\varepsilon} + n)$
[SS11]	semialgebraic (emptiness)	—	$O(t(\zeta^{-1}(n))^{1+\varepsilon} + n)$

Table 1: Partition theorems, their respective range searching and our results compared to those of [KMS24]. The columns show: cite of the plaintext partition theorems (1^{st} column); type of ranges (2^{nd} column); time to answer a range query as in[KMS24] (3^{rd} column); and time using our tightened theorem (4^{th} column). The rows are: Axis-parallel hyper-boxes (1^{st} row); halfspaces bounded by hyperplanes ranges (2^{nd} row); general semi-algebraic ranges (3^{rd} and 4^{th} rows). In the 3^{rd} row $c = d$, when $d \leq 4$ and $c = 2d - 4$ otherwise. The 4^{th} row achieves better bounds in plaintext, however, tightening its partition theorem is left for future work. The 5^{th} and 6^{th} rows refer to emptiness queries and were not mentioned in [KMS24]: halfspaces bounded by hyperplanes (5^{th} row); and ranges whose lower envelope of r ranges can be decomposed into $\zeta(r)$ elementary cells (6^{th} row). In general $\zeta(r) = O(r^c)$, where c is as above, but the authors of [SS11] improved it for a few special cases: e.g., α-fat triangles, circular-caps, segment-emptiness amid balls and convex 3d-shapes. Here, d is the dimension of the problem, n is the number of points in the database and t the time to compare a point to a query.

In the 1-dim case this yields a solution that answers a range searching query in $O(t \cdot n^{\varepsilon} + n)$, where t is the time to check whether $p \in \gamma$, and $\varepsilon > 0$ is a parameter. As usual in these cases, as ε is chosen to be smaller the multiplicative factor hidden by the $O(\cdot)$ notation grows. In this work we improve the running time of range searching by using a slightly different algorithm that required us to develop a variation to copy-and-recurse that we call *leveled copy-and-recurse*.

In the general d-dim case, the technique of [KMS24] results in a running time of $O(t \cdot n^{1-1/d+\varepsilon} + n^{1+\varepsilon})$. This has an overhead term of $O(n^{1+\varepsilon})$ that comes from a pre-processing step that turned the tree structure they used to a full complete tree by adding "dummy" nodes and increased the size of the database to $O(n^{1+\varepsilon})$. In this work we show a more efficient way to turn trees to complete full trees. This requires us to prove tighter versions of the partition theorems of [Mat91, AM94] which yield solutions with improved overhead $O(t \cdot n^{1-1/c+\varepsilon} + n)$, where $c = d$ when $d \leq 4$ and $c = 2d - 4$ when $d > 4$. The exponent of n is worse than that of [KMS24] when $d > 4$ because our improvement does not apply to the partition theorem in [AMS13]. Nevertheless, the improvement in the overhead from $O(n^{1+\varepsilon})$ to $O(n)$ is significant because the overhead of [KMS24] dominated the running time for some values of n and ε. Our results are summarized in Table 1.

In some cases we are only interested in answering an emptiness query, i.e., whether $P \cap \gamma = \emptyset$. Intuitively this is an easier task than computing $|P \cap \gamma|$ and indeed it was shown in [Mat91] that it can be answered in plaintext in $O(n^{1-1/\lfloor d/2 \rfloor+\varepsilon})$ time if γ is a halfspace bounded by a hyperplane in $\mathbb{R}^d$. This was later generalized to more types of ranges in [SS11]. In this paper we show how to implement these algorithms under FHE with only linear overhead.

Emptiness queries are used in many applications and we name here a few.

Checking a blacklist. A use case we consider is to preprocess a list $L_1, \ldots, L_n \in \mathbb{N}$ so we can efficiently answer whether a query $q \in \mathbb{N}$ is in the list. For example, the list can be of malicious IPs, maintained by one party and queried by another where both want to keep their data secret. In this case we can replace a query $q \in \mathbb{N}$ with the range $[q - 0.5, q + 0.5]$ and check whether the range is empty or not.

Approximate counting with a multiplicative factor. Given a set P and $\delta > 0$, the problem of approximate counting is to preprocess P such that given γ we can efficiently compute μ_γ such that $(1-\delta)|P \cap \gamma| \leq \mu_\gamma \leq (1+\delta)|P \cap \gamma|$. This is useful for example when estimating conditional probability. See Appendix C for more details.

Empty α-fat triangle in the plane. An α-fat triangle is a triangle with all angles greater than

α, for some $\alpha > 0$. Most polygons emanating from real life problems can be triangulated into α-fat triangles with some parameter $\alpha > 0$. When that is possible a set of points P on a map can be preprocessed so given a query polygon γ we can efficiently answer whether it contains any $p \in P$ by triangulating γ into α-fat triangles and checking each triangle for emptiness.

1.1 Our Contribution

- *Improved overhead.* We improve the overhead term for range searching given in [KMS24]. Specifically, we reduce the overhead from $O(n^{1+\varepsilon})$ to $O(n)$. For example, reducing the query time from $O(t \cdot n^{1-1/d+\varepsilon} + n^{1+\varepsilon})$ to $O(t \cdot n^{1-1/d+\varepsilon} + n)$ when the queries are halfspaces bounded by hyperplanes in $\mathbb{R}^d$. See rows 2-4 in Table 1.

  In more details, the larger overhead of [KMS24] was a result of $FillTree$ - a procedure that added "dummy" nodes to the tree until the tree becomes full. The $FillTree$ described by [KMS24] added $O(n^{1+\varepsilon})$ nodes. To improve this we give a tighter version of the partition theorems of [Mat91, AM94] together with a more economic filling technique that leaves the tree size at $O(n)$.

- *Leveled copy-and-recurse.* We consider a different version of copy-and-recurse where there is a bound ξ for the number of nodes in each level of the tree that the plaintext algorithm visits. This is different than the original copy-and-recurse in [KMS24] that requires that for each node, v, there is a bound $\xi = O(r^c)$ on the number of children that the plaintext algorithm visits.

  Leveled copy-and-recurse improves:

  - binary search (or binary trees) under FHE
  - traversing B-trees under FHE
  - 1-dimensional range searching

  to run in time $O(t \cdot \log n + n)$, where t is the time to compute (under FHE) the indicator whether the algorithm should recurse into a child. See row 1 in Table 1.

- *Emptiness queries.* We show how to answer emptiness queries (where the goal is to decide whether the query range is empty or not) faster than counting queries by showing how the partition theorems of [Mat92, SS11] can be used under FHE. This yields several cases of emptiness queries, for example: (1) halfspaces bounded by hyperplanes in time $O(t \cdot n^{1-1/\lfloor d/2 \rfloor + \varepsilon} + n)$ (instead of previously known $O(t \cdot n^{1-1/d+\varepsilon} + n^{1+\varepsilon})$); (2) α-fat 2-dim triangles $O(t \cdot n^{\varepsilon} + n)$ (instead of previously known $O(t \cdot n^{1-1/2+\varepsilon} + n^{1+\varepsilon})$). See rows 5-6 in Table 1.

- *Approximate counting.* We show how to use under FHE the work of [AHP08], that uses multiple emptiness queries to approximate counting query.

- *Implementation.* We implemented our algorithms using HElayers [AAB+20] and HEaaN [Cry22] and provide a comparison for the community to evaluate our algorithms. Our experiments show that our algorithms are faster than those of [KMS24]. For example, for a database of 2^{25} points our emptiness algorithm is $\times 3.6$ faster than what was known before.

2 Preliminaries and Related Work

In this section we review some preliminaries that are needed to understand our results. First we review preliminaries from computational geometry: emptiness queries in plaintext and how these queries are solved with parition trees. Then we review cryptographic preliminaries: fully homomorphic encryption (FHE) and the limitation of operating in circuit model. We then review some building blocks such as comparisons under FHE and selection matrices.

Number Representation When describing our algorithms we use real numbers. Computers use number representations such as floating points [iee08] or rational [NM90] but they cannot truly represent real numbers. In this paper we are not concerned with how numbers are represented and require only the existence of addition and multiplication operations.

Trees The data structures we review and use in our solution are based on trees. When describing a tree we use v to denote a node in a tree. We use dot (".") to denote members of v, so for example, $v.child[1], \ldots, v.child[r]$ are the children of v. The height of a node v is the number of nodes on the path from v to the root and the height of a tree is the maximal height of its nodes. In addition, as in [KMS24], we associate a subset of points to each node. We use $S(v)$ to denote the subset that is associated with the node v. Note that $S(v)$ is associated with v but is **not stored** in v which is why we avoid the dot notation.

2.1 Computational Geometry

Range Searching. A *range space* is a pair (X, Γ), where X is a set and $\Gamma \subset 2^X$ (i.e., each $\gamma \in \Gamma$ is a subset of X). Usually, $X = \mathbb{R}^d$ and Γ is a family of semi-algebraic sets with **constant description complexity**. That means that each $\gamma \in \Gamma$ is a union and intersection of a constant number of polynomial inequalities of constant degree. For example, X can the the 3D-space ($X = \mathbb{R}^3$) and Γ be the set of all 3D spheres.

α-fat triangles. A triangle on the plane is α-fat if all its angles are bigger than α. Fat triangles are interesting because in most real-life applications polygons can be triangulized into α-fat triangles for some $\alpha > 0$.

Simplicial partition. For a set of n points $P \subset \mathbb{R}^d$, a simplicial partition of size m of P is $\Pi = \{(P_1, \sigma_1), \ldots, (P_m, \sigma_m)\}$, where $P_1, \ldots, P_m \subset \mathbb{R}^d$ are a partition of P (that is, $\cup P_i = P$ and $P_i \cap P_j = \emptyset$ when $i \neq j$) and $\sigma_1, \ldots, \sigma_m \subset \mathbb{R}^d$ are simplices such that $P_i \subset \sigma_i$.

Crossing number. Given a simplicial partition Π and a range γ, we say that: (1) γ *contains* a simplex σ if $\sigma \subset \gamma$; (2) γ and σ are *disjoint* if $\sigma \cap \gamma = \emptyset$; otherwise, (3) we say that γ *crosses* σ.

The *crossing number* of γ with respect to Π is the number of simplices that γ crosses. When Π is obvious from the context we simply call this number the crossing number of γ.

Partition theorem. Given a family of ranges Γ, previous work (e.g., [Mat91, AMS13, SS11]) show how to partition P into a simplical partition of size $O(r)$ with $|P_i| = O(\frac{n}{r})$ such that:

- If the ranges in $\Gamma \subset 2^{\mathbb{R}^d}$ can be described with constant number of parameters then the crossing number of $\gamma \in \Gamma$ is $O(r^{1-1/d})$. See [AMS13].
- If $\gamma \subset \mathbb{R}^d$ is a half-space bounded by a hyperplane and $P \cap \gamma = \emptyset$ then the crossing number of γ is $O(r^{1-1/\lfloor d/2 \rfloor})$. See [Mat91].
- If $\gamma \subset \mathbb{R}^2$ is a α-fat triangle and $P \cap \gamma = \emptyset$ then the crossing number of γ is $O(r^{\varepsilon})$. See [SS11] (also see [SS11] for more partition theorems).

2.2 Emptiness Queries in Plaintext

In the range emptiness problem we are given a set of n points $P \subset \mathbb{R}^d$, and a family of *ranges* $\Gamma \subset 2^{\mathbb{R}^d}$. The goal is to build a data structure such that given a range $\gamma \in \Gamma$ we can quickly determine whether $P \cap \gamma = \emptyset$. Range counting is a related problem. Indeed, given the count $|P \cap \gamma|$ we can determine that $P \cap \gamma = \emptyset$ if $|P \cap \gamma| = 0$.

In this paper we are interested in data structures of linear size and a sub-linear query time. In a seminal work, Matoušek [Mat91] showed that if Γ is the family of all halfspaces that are bounded by hyperplanes, a linear data structure can be constructed that answers the range counting problem in $O(n^{1-1/d+\varepsilon})$ time. This work was later generalized in [AMS13] to answer range counting for general semi-algebraic ranges with the same time bound.

Intuitively, answering emptiness queries should be more efficient. Indeed, Matoušek [Mat92] showed that for halfspaces bounded by hyperplanes emptiness queries can be answered in $O(n^{1-1/\lfloor d/2 \rfloor+\varepsilon})$ time, using a linear data structure. For general ranges, the work of [AM94] shows how to lift the problem to a higher dimension where the ranges are mapped to hyperplanes and then the work of [Mat92] can be used. The question of answering range emptiness without lifting was left unanswered for more than 2 decades until Sharir and Shaul [SS11] showed a non-trivial solution. The running time they derived depends on the size of the decomposition of the lower envelope of a random sample of r ranges. In the general case this leads to the same time bound as [AM94] because the best known bound on the size of that decomposition is the same as the decomposition of the entire arrangement of ranges. Nevertheless, [SS11] showed better bounds for a few cases:

1. Fullness queries amid points in 3D where the ranges are convex objects of a certain type e.g., cylinders, spheres, etc. For example, these queries can be used to determine whether all points lie within a certain distance from a query line or a query point.

   Theorem 1 (Theorem 4.2 in [SS11]). *Let P be a set of n points in $\mathbb{R}^3$, and let Γ be a family of convex ranges of constant description complexity. Then one can construct, in near linear time, a data structure of linear size so that, for any range $\gamma \in \Gamma$, it can determine, in $O(n^{1/2+\varepsilon})$ time, whether γ is full.*

2. α-fat triangle emptiness on the plane. This query can be used to determine whether a query polygon that can be triangulized into $O(1)$ α-fat triangles is empty.

   Theorem 2 (Theorem 6.1 in [SS11]). *One can preprocess a set P of n points in the plane, in near-linear time, into a data structure of linear size, so that, for any query α-fat triangle Δ, one can determine, in $O(n^{\varepsilon})$ time, whether $\Delta \cap P = \emptyset$.*

All these linear-size data structures are based on Partition trees which we review next. The different running times come from different partition theories which we briefly explain in the next section.

2.3 Partition Trees

As hinted in the previous section, the results of [Mat91, Mat92, AMS13, SS11] are based on a data structure called partition tree which we review now.

A partition tree, T, for a set $P \subset \mathbb{R}^d$ is built as follows. The root of T represents the entire set P, i.e., $S(root) = P$, where *root* is the root of T. Additionally, the root keeps a bounding simplex $\sigma \subset \mathbb{R}^d$ (i.e., $P \subset \sigma$). The set P is then partitioned into a simplicial partition $\{(P_1, \sigma_1), \ldots, (P_m, \sigma_m)\}$ with $P_1, \ldots, P_m$ having "almost" the same size. The root then has m children, $root.child[1], \ldots, root.child[m]$, where $S(root.child[i]) = P_i$ and we recursively partition each P_i. The recursion stops at nodes (leaves) that each represents a single point $p \in P$.

Searching A Partition Tree. We now show how to use a partition tree with a query range γ, for example to compute $|P \cap \gamma|$. We start at the root and go over its children. For $root.child[i]$ that represents $S(root.child[i]) = P_i \subset \sigma_i$ we compare σ_i to γ and check whether it is (1) contained in γ, i.e., $\sigma_i \subset \gamma$; (2) disjoint, i.e., $\sigma_i \cap \gamma = \emptyset$ or (3) σ_i crosses γ. In case (1) we know that $P_i \subset \sigma_i \subset \gamma$ and we count all the points of P_i. In case (2) we know that none of the points in P_i is in γ and we ignore this child and its subtree. In the last case (3), we need to recurse into $root.child[i]$ and compare a finer partition of P_i with γ. The ingenuity of [Mat91] was to show how to construct a simplicial partition of P such that any range $\gamma \in \Gamma$ crosses at most $O(m^{1-1/d})$ simplices. In his seminal work Matoušek considered only halfspaces bounded by a hyperplane. This was later (see [AM94, AMS13]) generalized to ranges with constant description complexity. This is summarized in the following partition theorem.

Theorem 3 (From [AMS13]). *Given a set P of n points in $\mathbb{R}^d$, for some fixed d, a family of semi-algebraic ranges of constant description complexity Γ and a parameter $r \leq n$, a simplicial partition $\Pi = \{(P_1, \sigma_1), \ldots, (P_m, \sigma_m)\}$ can be computed such that:*

1. *$\lfloor n/r \rfloor \leq |P_i| < h\lfloor n/r \rfloor$ for every i and some constant h.*
2. *The crossing number of any $\gamma \in \Gamma$ is $O(r^{1-1/d})$.*

In a parallel work Matoušek [Mat92] showed that if we are interested only in emptiness queries then P can be partitioned into a simplicial partition such that an empty range $\gamma \in \Gamma$ crosses at most $O(m^{1-1/\lfloor d/2 \rfloor})$ simplices. Matoušek showed this only for halfspaces bounded by hyperplanes. Later, Sharir and Shaul [SS11] showed how to generalize this to more ranges.

Theorem 4 (Summarizing [Mat92] and [SS11]). *Given a set P of n points in $\mathbb{R}^d$, for some fixed d, a family of semi-algebraic ranges Γ and a parameter $r \leq n$, a simplicial partition $\Pi = \{(P_1, \sigma_1), \ldots, (P_m, \sigma_m)\}$ can be computed such that $\lfloor n/r \rfloor \leq |P_i| < h\lfloor n/r \rfloor$ for every i and some constant h and the crossing number of $\gamma \in \Gamma$ is as follows:*

- *If $P \subset \mathbb{R}^d$, Γ is the family of all halfspaces bounded by hyperplanes and $P \cap \gamma = \emptyset$ then the crossing number of Π is $O(r^{1-1/\lfloor d/2 \rfloor})$ (see [Mat92]).*
- *If $P \subset \mathbb{R}^2$, Γ is the family of all α-fat triangles and $P \cap \gamma = \emptyset$ then the crossing number of Π is $O(r^{\varepsilon})$ (see [SS11]).*
- *If $P \subset \mathbb{R}^3$ and Γ is the family of convex shapes of constant description complexity and $P \cap \gamma = P$ then the crossing number of Π is $O(r^{2/3})$ (see [SS11]).*

As an example, we describe a partition tree built with parameter r for a set of 1D points $P = \{p_1, \ldots, p_n\} \subset \mathbb{R}$. For simplicity assume $|P| = n$ is a power of r. Also, assume $p_1 \leq p_2 \leq \cdots \leq p_n$. To build the tree T we:

1. partition P into $\{(P_1, \sigma_1), \ldots, (P_r, \sigma_r)\}$ (i.e. $m = r$) by setting
$$P_i = \{p_{1+(i-1)r}, p_{2+(i-1)r}, \ldots, p_{r+(i-1)r}\},$$
and
$$\sigma_i = [p_{1+(i-1)r}, p_{r+(i-1)r}].$$
2. add r children to the root of T, where $S(root.child[i]) = P_i$.
3. recurse into each child until we have nodes corresponding to a single point.

See Figure 2 for a depiction of a 1D partition tree.

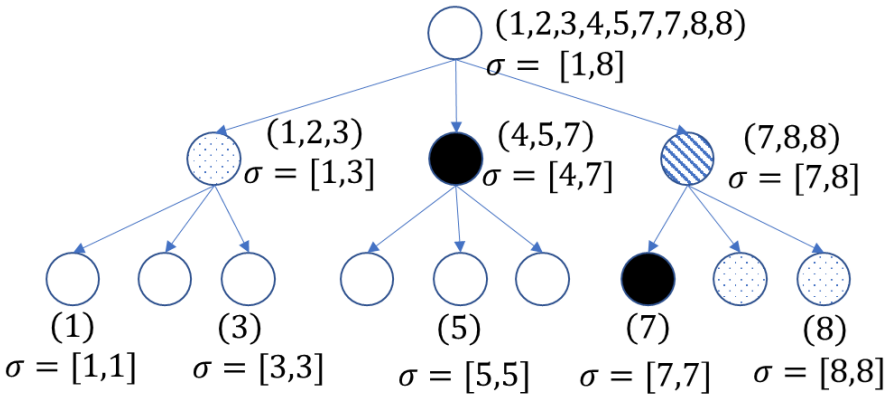

Figure 2: An example of a 1-dim partition tree that was built for the numbers 1,2,3,4,5,7,7,8,8 with $r = 3$. At each node v we show S_v and the bounding segment of S_v (for readability we omit this for some leaves). The figure also shows how the tree is traversed for counting all values in a query segment $\gamma = [4, 7]$. Nodes with $v.\sigma \subset \gamma$ are marked with **solid black**. All values in these nodes can be counted without recursing. Nodes with $v.\sigma \cap \gamma = \phi$ are marked with **black dots**. All values in these nodes can be ignored. Nodes the traversal doesn't reach (because an ancestor is one of the cases above) are marked with **white**. Nodes with $v.\sigma$ crossing γ are marked with **black stripes**. These nodes should be recursed into to determine how many of the values they represent are in γ.

It is easy to see that for any $\gamma \subset \{\text{all segments in } \mathbb{R}\}$ and a partition $\{(P_1, \sigma_1), \ldots, (P_r, \sigma_r)\}$, γ crosses at most 2 simplices.

Lemma 1. *[Lemma 6.1 in [KMS24]] Let $\sigma_1 = [\sigma_{min_1}, \sigma_{max_1}], \ldots, \sigma_r = [\sigma_{min_r}, \sigma_{max_r}]$ be r segments, where $\sigma_{min_1} \leq \sigma_{max_1} \leq \ldots \leq \sigma_{min_r} \leq \sigma_{max_r}$. Then, a segment $\gamma = [\gamma_{min}, \gamma_{max}]$ crosses at most 2 segments of $\sigma_1, \ldots, \sigma_r$.*

It is also easy to see that if γ is empty then γ crosses at most 1 bounding simplex

Lemma 2. *Let $\{(P_1, \sigma_1), \ldots (P_r, \sigma_r)\}$ be a partition of P as in Lemma 1 then, if $P \cap \gamma = \emptyset$ then γ crosses at most 1 segment of $\sigma_1, \ldots, \sigma_r$.*

Proof. Assume that γ crosses 2 bounding segments σ_i and σ_j. Then
$$\sigma_{min_i} < \gamma_{min} \leq \sigma_{max_i} \leq \sigma_{min_j} \leq \gamma_{max} < \sigma_{max_j}.$$
This means that $\sigma_{max_i}, \sigma_{min_j} \in \gamma$ which contradicts the emptiness of γ. □

2.4 Fully Homomorphic Encryption

Our algorithms and protocols can be implemented using FHE or other MPC schemes that support addition and multiplication. However, to enhance clarity, we present our protocols using FHE.

FHE (see, e.g., [BGV12, CKKS17]) is an asymmetric encryption scheme that supports both addition ($+$) and multiplication ($\times$) operations on ciphertexts. Specifically, an FHE scheme is defined by the tuple $\mathcal{E} = (Gen, Enc, Dec, Add, Mult)$, where:

- $Gen(1^\lambda, p)$ gets a security parameter λ and an integer p and generates the keys pk and sk.
- $Enc_{pk}(m)$ gets a message m and outputs a ciphertext $[\![m]\!]$.
- $Dec_{sk}([\![m]\!])$ gets a ciphertext $[\![m]\!]$ and outputs a message m'.
- $Add_{pk}([\![a]\!], [\![b]\!])$ gets two ciphertexts $[\![a]\!], [\![b]\!]$ and outputs a ciphertext $[\![c]\!]$.
- $Mult_{pk}([\![a]\!], [\![b]\!])$ gets two ciphertexts $[\![a]\!], [\![b]\!]$ and outputs a ciphertext $[\![d]\!]$.

The scheme is considered **correct** if $m = m'$, $c = a + b \mod p$, and $d = a \cdot b \mod p$. In approximated FHE schemes (e.g., CKKS [CKKS17]), correctness is relaxed to $m \approx m'$, $c \approx a + b$, and $d \approx a \cdot b$.

Semantic security requires that given pk and a set of ciphertexts $[\![m_1]\!], \ldots, [\![m_{poly(\lambda)}]\!]$, and their corresponding plaintexts $m_1, \ldots, m_{poly(\lambda)}$, where $poly(\lambda)$ is polynomially dependent on λ, the probability of deducing m_0 from another ciphertext $[\![m_0]\!]$ should be negligible in λ.

By leveraging addition and multiplication, we can construct any arithmetic circuit and evaluate any polynomial $\mathbb{P}(x_1, \ldots)$ on ciphertexts $[\![x_1]\!], \ldots$. For example, in a client-server setup, the client encrypts her data and sends it to the server, which computes a polynomial $\mathbb{P}$ on the encrypted inputs. The server then returns the encrypted result to the client, who decrypts it. FHE's semantic security guarantees that the server gains no information about the client's data.

When evaluating an arithmetic circuit, we are concerned with the size of C, denoted size(C), which is the number of operators in C and with the depth of C, denoted depth(C), which is the maximal number of multiplication gates on a path of C. The time to evaluate a circuit is then $Time = overhead \cdot \text{size}(C)$, where in many schemes $overhead$ grows when depth(C) increases.

Abbreviated syntax To make our algorithms and protocols more intuitive to read we use $[\![\cdot]\!]_{pk}$ to denote a ciphertext. When pk is clear from the context we omit it. We use an abbreviated syntax:

- $[\![a]\!] + [\![b]\!]$ is short for $Add_{pk}([\![a]\!], [\![b]\!])$.
- $[\![a]\!] \cdot [\![b]\!]$ is short for $Mult_{pk}([\![a]\!], [\![b]\!])$.
- $[\![a]\!] + b$ is short for $Add_{pk}([\![a]\!], Enc_{pk}(b))$.
- $[\![a]\!] \cdot b$ is short for $Mult_{pk}([\![a]\!], Enc_{pk}(b))$.

Challenges of FHE The plaintext algorithms in Sections 2.2 and C.1 work in comparison model, where they make decisions based on the input. Algorithms under FHE work in circuit model where the operations they make are not dependent on the input. For example, under FHE it is impossible to have "if" statements and branches in the code. A general recipe to convert an algorithm from comparison model to circuit model is to replace every "if" statement and its "then" and "else" branches with a subcircuit that computes a ciphertext indicating whether the condition of the "if" statement holds and use it with a multiplexer (MUX) gate to choose between the outputs of the two branches. It easily follows that using this recipe on an algorithm that traverses a tree leads to an algorithm that is no better than naïve algorithm that goes over all the leaves.

2.5 Comparisons Under FHE

A key ingredient in the recipe to convert an algorithm in comparison model to circuit model is a subcircuit that computes a ciphertext that holds the encryption of 1 when a condition holds and 0 otherwise. Computing these conditions is done using a smaller building block that computes an encrypted indicator for the comparison of 2 ciphertexts $IsSmaller(x, y) = \begin{cases} 1, & \text{if } x < y \\ 0, & \text{otherwise.} \end{cases}$. This problem was considered in the past. For example, see [CKK20] for the approximated setting of CKKS. The implementation details of $IsSmaller$ depend on the underlying FHE scheme and the way numbers are represented. With this primitive, more complicated tests can be constructed, e.g., whether a value x is contained inside the segment $[a, b]$.

2.5.1 Making Tests Under FHE

In this paper we assume the existence of 2 functions $IsContaining$ and $IsCrossing$ (described in detail below). The implementation of these functions depends on the range shapes, the FHE scheme and the number representation. In case the FHE scheme is approximated (CKKS) it also depends on the desired accuracy. Our protocols use these functions as black-boxes and we are not concerned with how they are implemented. Furthermore, we express the complexity bounds of our protocol with respect to $IsContaining$ and $IsCrossing$ (see below the definition of t and s, parameters that capture the "hardness" of these functions).

For example in the 1-dim case, these functions are (see also Figure 3):

- $IsContaining([\![\sigma]\!], [\![\gamma]\!])$. This function gets as input two encrypted segments $\sigma, \gamma \subset \mathbb{R}$. The output of $IsContaining$ is a ciphertext $[\![c]\!]$, where $c = 1$ if $\sigma \subseteq \gamma$ and $c = 0$ otherwise.
- $IsCrossing([\![\sigma]\!], [\![\gamma]\!])$. This function gets as input two encrypted segments $\sigma, \gamma \subset \mathbb{R}$. The output of $IsCrossing$ is a ciphertext $[\![c]\!]$, where $c = 1$ if γ crosses σ (i.e. intersects but not contains) and $c = 0$ otherwise.

The d-dimension version of these functions is similar except that it gets a simplex $\sigma \subset \mathbb{R}^d$ and a range $\gamma \subset \mathbb{R}^d$.

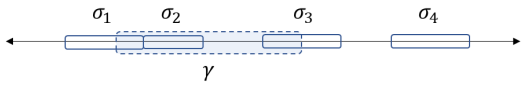

Figure 3: An example of 1 dimensional bounding segments $\sigma_1, \ldots, \sigma_4$ and a range γ. In this example γ contains σ_2 and crosses σ_1 and σ_3.

2.6 Size and Depth of Computing $IsContaining$ and $IsCrossing$ (t and s)

We define t and s as the size and depth of the sub-circuit that computes $IsContaining$ and $IsCrossing$. These parameters capture the "hardness" of comparing a range to a simplex in FHE.

Note that the implementation of $IsCrossing$ and $IsContaining$ varies depending on the FHE scheme and the way numbers are represented, as well as on the primitive $IsSmaller$ on which they rely. In general, we consider high-dimensional settings, where the implementation of these functions also depends on the shape of the query ranges.

In approximate schemes, the implementations of $IsContaining$ and $IsCrossing$ are also influenced by $n = |P|$. As n grows, these functions need to be more accurate since any error is compounded over more points. Consequently, $s = s(n)$ and $t = t(n)$ are functions of n. The specific details depend on the implementation and the parameters of the cryptographic scheme.

In practice, most of the runtime in range searching is spent executing these functions. By expressing the runtime as a function of t and s, which reflect the complexity of $IsContaining$ and $IsCrossing$, we get a clearer way to compare different range-searching algorithms.

2.7 Selection matrix

Given a binary vector $\chi \in \mathbb{R}^d$ (often called an *indicator vector*) whose Hamming weight is $\xi < d$ (i.e., there are ξ non-zero elements). A selection matrix $M(\chi) \in \{0,1\}^{\xi \times d}$ is a matrix that has the property that it *selects* elements indicated by χ. That is, if $i_1, \ldots, i_\xi$ are the indexes of the non-zero elements in χ then for any vector $x = (x_1, \ldots, x_d)$ we have $M \cdot x^T = (x_{i_1}, \ldots, x_{i_\xi})$. Intuitively, a selection matrix is a generalization of the *multiplexer* (Mux) operation that selects one of 2 inputs given a binary selector value. Algorithm 1 in [KMS24] shows how to construct a selection matrix using a circuit of size $O(\xi \cdot d^2)$ and depth $O(\xi \cdot \log d)$. In a nutshell, to construct M we set $M_{r,c} = 1$ if the c-th element in x is the r-th non-zero element, otherwise we set $M_{r,c} = 0$.

2.8 Copy and Recurse

Copy and recurse is a technique from [KMS24] to traverse a tree efficiently under FHE. with this technique a plaintext algorithm can be converted to run under FHE by paying an overhead linear in the tree size. More specifically, given a tree T with m leaves and with these prerequisites:

1. every inner node of T has r children
2. every leaf of T is at the same distance from the root

and a plaintext algorithm $\mathcal{A}$ that:

1. when traversing T and reaching an inner node, $\mathcal{A}$ continues in at most $\xi \leq r^c$ children, where $c < 1$ is a constant
2. at every node v that $\mathcal{A}$ visits it computes a function $f(v)$ which takes time t to compute

then using Copy and Recurse, $\mathcal{A}$ can be implemented under FHE in time $O(t \cdot m^{c+\varepsilon} + m)$. The main idea behind copy and recurse is to: (1) determine (under FHE) at each inner node v which $\xi < r$ children of v $\mathcal{A}$ would have recursed into; (2) generate a selection matrix, M, that selects those ξ children; (3) multiply M by the vector of children to copy them and their subtrees (again under FHE); and then (4) recurse into those copies. The trick here is that although copying the ξ children takes $O(m)$ time under FHE, after doing so the number of times f is computed is similar to that of $\mathcal{A}$. See Figure 4 for a depiction.

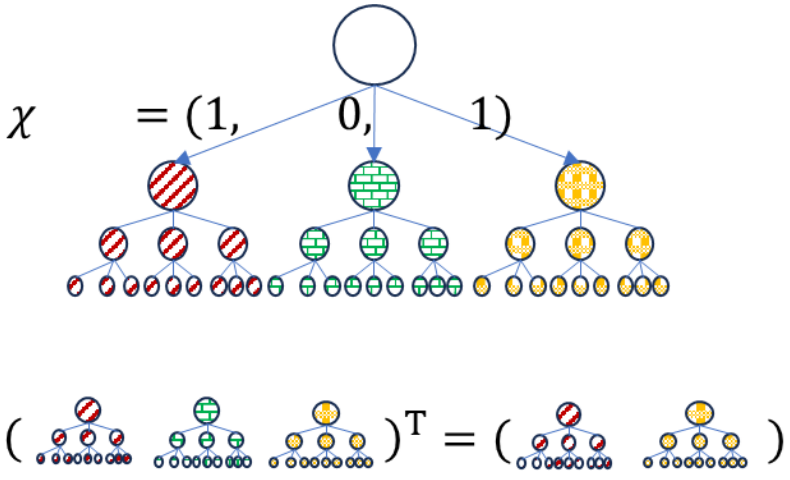

Figure 4: A depiction of copy-and-recurse. The top shows a 3-ary tree with an indicator vector χ indicating that the 1st and 3rd children need to be recursed into. The bottom shows how multiplying the selection matrix X_χ by the vector of 3 sub-trees results in the 2 sub-trees that need to be recursed into.

3 Problem Statement

In this paper we consider two related problems. First we consider the range searching problem that was considered in [KMS24] and improve their results. Then we consider the related range emptiness problem.

Range Searching Queries Given a set of n points $P \subset \mathbb{R}^d$ and a family of ranges $\Gamma \subset 2^{\mathbb{R}^d}$ we want to construct a linear data structure such that given a range $\gamma \in \Gamma$ we want to compute $f(P \cap \gamma)$ as quickly as possible, where f is a function that compute computes something on a subset of $\mathbb{R}^d$.

Emptiness Queries Given a set of n points $P \subset \mathbb{R}^d$ and a family of ranges $\Gamma \subset 2^{\mathbb{R}^d}$ we want to construct a linear data structure such that given a range $\gamma \in \Gamma$ we want to answer whether $P \cap \gamma = \emptyset$ as quickly as possible.

3.1 Threat Model

Our work is motivated by the trend toward cloud-based computation to reduce IT costs. Companies like IBM [ibm24], CryptoLab [hea24], and Duality [dua24] offer analytics-as-a-service, highlighting this shift.

We adopt the threat model of [KMS24], involving three parties: (1) a data owner with dataset P, (2) a querier performing range emptiness or approximate counting queries, and (3) the cloud, which hosts the database and executes the computations. Protocol 1 shows an overall view of a system implementing our secure emptiness searching protocol. The protocol for range searching is similar and is identical to Protocol 2 in [KMS24].

We consider computationally-bounded, semi-honest adversaries. We assume the querier doesn't collude with the cloud or the data owner. The semantic security of FHE guarantees that the cloud learns nothing on the content of γ or P.

Multiple Queriers. Protocol 1 considers a single querier. The protocol can be extended to multiple queriers in a similar way to Protocol 2 of [KMS24]. Their extention used transciphering (e.g., [ADE+23, TCBS23]) to avoid keeping an encrypted version of the database for each querier.

4 Improving Range Searching Overhead

The results of [KMS24] include an overhead of $O(n^{1+\varepsilon})$, for example, for ranges that are halfspaces bounded by hyperplanes, [KMS24] showed range searching can be answered in $O(t \cdot n^{1-1/d+\varepsilon} + n^{1+\varepsilon})$. In this section we address the overhead term and show how to reduce it to linear, i.e., the above range searching problem can be answered with a circuit of size only $O(t \cdot n^{1-1/d+\varepsilon} + n)$. In what follows we first remind why the overhead in [KMS24] was $O(n^{1+\varepsilon})$ and then show how to reduce it to $O(n)$ (which is also the lower bound). To do that we need to revisit Matoušek's partition theorem [Mat91] and prove a tighter version of it. Our tighter version improve the overhead of all partitions trees that are built using partition theorems whose proof is similar to that of [Mat91] (e.g., [Mat92, SS11, AM94]). Our improved overhead does not apply to the partition theorem in [AMS13] whose proof is different (specifically, it uses tools from algebraic geometry).

The discussion below addresses the partition theorem given in [Mat91]. Later we mention how to tighten the partition theorems in [Mat92, SS11, AM94] in a similar manner.

4.1 Where The $O(n^{1+\varepsilon})$ Overhead Comes From

As mentioned in [KMS24], the partition theorem of [Mat91] states that a set with n points can be partitioned into $\Pi = ((P_1, \sigma_1), \ldots, (P_m, \sigma_m))$, where $n/2r \leq |P_i| \leq n/r$ (and therefore $r \leq m \leq 2r$), for any parameter r.

Specifically, the sizes of P_i are roughly, **but not exactly**, the same. This is further exacerbated when using the partition theorem recursively to create a partition tree. In the extreme case consider a tree whose root has 2 subtrees T_1 and T_2 each having n leaves. Then let $S(v_1)$ (the points associated with v_1) of every $v_1 \in T_1$ be partitioned to $2r$ subsets $((P_1^{v_1}, \sigma_1^{v_1}), \ldots, (P_{2r}^{v_1}, \sigma_{2r}^{v_1}))$ and $|P_i^{v_1}| = \frac{|S(v_1)|}{2r}$. Similarly, let $S(v_2)$ of every $v_2 \in T_2$ be partitioned to r subsets $((P_1^{v_2}, \sigma_1^{v_2}), \ldots, (P_r^{v_2}, \sigma_r^{v_2}))$ and $|P_i^{v_2}| =$

Protocol 1: *RangeEmptinessProtocol*

Parties: Data owner, Querier, Cloud.

Parameters: $n, d > 0$; $\Gamma \subset 2^{\mathbb{R}^d}$.

Data owner input: A set $P \subset \mathbb{R}^d$ of n points.

Querier input: A pair (sk, pk) of secret and public keys.
Ranges $\gamma_1, \ldots, \gamma_c \in \Gamma$, where $c \in \mathbb{N}$ is polynomial in the security parameter of (sk, pk).

The cloud has no input.

Querier output: Indicators $E_1, \ldots, E_c$, where

$$E_i = \begin{cases} 1 & \text{if } P \cap \gamma_i = \emptyset \\ 0 & \text{otherwise} \end{cases}.$$

The cloud and the data owner have no output.

Querier performs:
  Send pk to the Cloud and to Data owner.
Data owner performs:
  Choose a parameter $0 < r < n$.
  $(T, \xi) :=$ Build a partition tree for P and Γ with parameter r. `// See Section 2.3`
  $[\![T]\!] :=$ encrypt $v.\sigma$ for every $v \in T$.
  Send $[\![T]\!], n, \Gamma, \xi, r$ to Cloud.
foreach $i = 1, \ldots, c$ **do**
  Querier performs:
    $[\![\gamma_i]\!] := \mathrm{Enc}_{pk}(\gamma_i)$.
    Send $[\![\gamma_i]\!]$ to Cloud.
  Cloud performs:
    $[\![E_i]\!] := PPRangeEmptiness_{n,d,\Gamma,\xi,r}([\![T]\!], [\![\gamma_i]\!])$. `// See Algorithm 2`
  `/* Similarly, for approx. counting Line 13 should call`
    $PPApproxCounting_{n,d,\Gamma,\xi,r}$ `(Section C)` `*/`
    Send $[\![E_i]\!]$ to querier.
  Querier performs:
    $E_i := \mathrm{Dec}_{sk}([\![E_i]\!])$.
    Output E_i.

$\frac{|S(v_2)|}{r}$. It easily follows that the height of T_1 is $\log_{2r} n$ and the height of T_2 is $\log_r n$. In plaintext, the difference in the shape of the subtrees does not affect the running time of range queries but under FHE this imbalance creates 2 problems that are already mentioned in [KMS24]: (1) the structure of the tree is not hidden and may leak information on P; (2) copy-and-recurse requires that all subtrees have the same structure.

The solution suggested by [KMS24] was to "fill" the tree by adding "dummy" nodes to the partition tree until it becomes full. Unfortunately, as [KMS24] showed, the total leaves in the full tree is upper bounded by $O(n^{1+\log_r 2}) = O(n^{1+\varepsilon})$.

4.2 A More Efficient Alternative To *FillTree*

We propose a more efficient way to generate full trees. Specifically with our alternative the number of leaves remain linear. To do that we first modify the partition theorem of [Mat91]. Our modified theorem assumes that n is a multiple of r, and in that case we show the subsets of the partition are of the same size: $|P_i| = n/r$. Next, we show that when n is a power of r, we can use the modified partition theorem to construct a full tree with n leaves. Then, we address the case when n is not a power of r. Finally, we describe how other partition theorems can be changed in a similar way.

4.3 Modifying Matoušek's Partition Theorem

We now give a modified version of the partition theorem (Theorem 3.1) of [Mat91]. The proof is almost identical to that of [Mat91] so we just sketch it in high level and mention where it diverges.

Theorem 5. *Let $P \in \mathbb{R}^d$ be a set of n points, let $r \in \mathbb{N}$ be a parameter that divides n. Also, let $\Gamma \subset 2^{\mathbb{R}^d}$ be the set of all hyperplanes in $\mathbb{R}^d$. There exists a simplicial partition $\Pi = ((P_1, \sigma_1), \ldots, (P_r, \sigma_r))$, that satisfies $|P_i| = n/r$ and whose crossing number is $O(r^{1-1/d})$.*

Specifically, there are 2 differences from Theorem 3.1 in [Mat91]: (1) we assume that n is a multiple of r and (2) $|P_i| = n/r$, for all i.

Proof Sketch of Theorem 5 The main part of the proof is Lemma 3 which is similar to Theorem 5 with the difference that the crossing number of Π is computed with respect to a finite set $Q \subset \Gamma$ (called the *test set*). Using this lemma, the proof of Theorem 5 is completed by showing how to construct Q where $|Q| = O(n^d)$ such that every range in $\gamma \in \Gamma$ has the same crossing number as Q (up to a constant multiplicative factor). We skip the construction of the test set Q and the proof of its properties since it is identical to the proof in [Mat91] and turn to prove the following lemma which is a modified version of Lemma 3.2 in [Mat91].

Lemma 3. *Let P, n, r be as above. Also, let $Q \subset 2^{\mathbb{R}^d}$ be a finite set of hyperplanes in $\mathbb{R}^d$. There exists a simplicial partition $\Pi = ((P_1, \sigma_1), \ldots, (P_r, \sigma_r))$, that satisfies $|P_i| = n/r$ and the crossing number of each $h \in Q$ is $O(r^{1-1/d} + \log |Q|)$.*

The proof of this lemma is almost identical to that of Lemma 3.2 in [Mat91]. In fact, in Matoušek's original proof $|P_1| = \cdots = |P_{m-1}| = n/r$ and only $n/r \leq |P_m| < 2n/r$, also $m \in \{r-1, r\}$. As we show, we use the assumption that n is a multiple of r to show that $|P_i| = n/r$ for all $1 \leq i \leq r$. Since the difference in the proofs is so small we only sketch it in high level.

Proof sketch Construct inductively the disjoint sets $P_i \subset P$ and their enclosing simplices. Suppose that $P_1, \ldots, P_i$ have already been constructed. Set $P'_i = P \setminus (P_1 \cup \cdots \cup P_i)$ and $n_i = |P'|$. If $n_i < 2n/r$, we set $P_{i+1} = P'_i$, $\sigma_{i+1} = \mathbb{R}^d$ and conclude the construction. Otherwise, for any $h \in Q$, let $\kappa_i(h)$ denote the number of simplices of $\sigma_1, \ldots, \sigma_i$ that h crosses. Let $t_i = O((r \cdot n_i/n)^{1/d})$ and construct the decomposition of the weighted $1/t_i$-cutting, Ξ_i, of Q, where the weight of any $h \in Q$ is $w_i(h) = 2^{\kappa_i(h)}$. Ξ_i has at most $r \cdot n_i/n$ cells. By the pigeonhole principle, there is at least one cell $\sigma_i \in \Xi_i$ that contains at least n/r points of P'_i. Choose arbitrarily n/r points $P_i \subset P' \cap \sigma_i$ (as already mentioned σ_i is the simplex in Ξ_i that contains them). Next, Matoušek proves that due to the exponential weights $w_i(h)$, the crossing number of any $h \in Q$ is at most $O(\log |Q| + r^{1-1/d})$. We do not repeat this proof here.

To our matter, Matoušek's construction yield $|P_1| = \cdots = |P_{r-1}| = n/r$. Only for the last set they get $n/r \leq |P_m| < 2n/r$, however since we assume n is a multiple of r (and this is where we deviate from Matoušek's original proof) it follows that $m = r$ and $|P_r| = n/r$ as required.

4.4 Constructing a Full Partition Tree

We now turn to explain how we use the modified partition theorem to build a partition tree that is also full. First we assume n is a power of r. Next, we give constructions for the case where n is not a power of r.

The case where n is a power of r When n is a power of r the partition tree can be trivially built as follows: Apply Theorem 5 on P. Since $|P| = n$ is divided by r we get a partition $((P_1, \sigma_1), \ldots, (P_r, \sigma_r))$ with $|P_i| = n/r$. Let the root have r children, associate the i-th child with P_i and continue in recursion for each child. Since n is a power of r, at each node v we have $|S(v)|$ a multiple of r. Specifically, if v is at level ℓ, then $|S(v)| = n/r^\ell$.

When n is not a power of r Set $m = r^{\lceil \log_r n \rceil}$ and add $m - n$ dummy nodes. In this case $m < r \cdot n$ and we continue as before.

Another option is to consider the unique base-r representation $n = \sum_i c_i \cdot r^i$, and have c_i trees of size r^i, for $i = 0, \ldots, \lfloor \log_r n \rfloor$. In this case a range search query is done separately on each tree and then aggregated together. For example, range counting when the ranges are half-spaces bounded by hyperplanes is done in time

$$\sum_i c_i \cdot O((r^i)^{1-1/d+\varepsilon} + r^i) = O(n^{1-1/d+\varepsilon} + n).$$

Other partition theorems that follow the same lines in the proof (e.g., [AM94, Mat92, SS11]) can be improved in the same way (see Appendix B for more details). We note that the proof of the partition theorem in [AMS13] is based on other techniques (from algebraic geometry) and the similar improvement of this theorem is left for future work.

We conclude with the corollary below (also restated and proved in Appendix B). The results are also summarized in Table 1.

Given a set $P \subset \mathbb{R}^d$ of n points, we can construct an encrypted data structure for a family of ranges Γ that given an encrypted range $\gamma \in \Gamma$ can answer emptiness queries in a circuit of size $T(n)$ and depth $O(s \cdot \log n)$, where:

1. when $P \subset \mathbb{R}^d$ is a set of points Γ is the family of all halfspaces bounded by a hyperplane, then we can answer half-space emptiness in a circuit of size $T(n) = O(t \cdot n^{1-1/\lfloor d/2 \rfloor + \varepsilon} + n)$.

2. When P is a set of n balls in 3-space and $\Gamma = \{\text{segments in 3-space}\}$, then we can answer whether a segment does not intersect any ball in a circuit of size $T(n) = O(t \cdot n^{2/3+\varepsilon} + n)$.

3. When $P \subset \mathbb{R}^3$ is a set of points and $\Gamma = \{\text{convex objects}\}$, then we can answer whether $\gamma \in \Gamma$ contains all the points in P (i.e., the complement to γ is empty) in a circuit of size $T(n) = O(t \cdot n^{1/2+\varepsilon} + n)$.

4. When $P \subset \mathbb{R}^2$ is a set of points and $\Gamma = \{\alpha\text{-fat trianlge}\}$ or $\Gamma = \{\text{large circular cap}\}$, then we can answer whether γ is empty in a circuit of size $T(n) = O(t \cdot n^{\varepsilon} + n)$.

5 Emptiness Under FHE

In this section we show how to solve the range emptiness problem. While range emptiness is interesting in its own, it is also used as a building block for other algorithms and protocols. See for example Appendix C for a protocol that answers the approximate range counting using multiple emptiness queries. Our solution to the emptiness problem is given in Algorithm 2.

Algorithm Overview *PPRangeEmptiness* gets parameters n, r, ξ, d and a family of ranges Γ. It also gets two inputs: (1) a partition tree $[\![T]\!]$ with n leaves built for a set of n points $P \subset \mathbb{R}^d$; and (2) a range $\gamma \in \Gamma$. *PPRangeEmptiness* works recursively and returns $[\![1]\!]$ if $P \cap \gamma = \emptyset$, and $[\![0]\!]$ otherwise.

The algorithm starts at the root of T (denoted by v). First the algorithm checks whether v is a leaf. If it is then we have reached the end of the recursion (Line 3). In this case $S(v) = \{p\}$ is a single point, $v.\sigma = p$ is a degenerate simplex and $P \cap \gamma = \emptyset$ if $p \notin \gamma$. This is checked in Line 3.

We now consider the case where v is not a leaf, i.e., v is a root of a subtree in T. In that case v has r children and $P \cap \gamma = \emptyset$ iff all these conditions hold:

1. there is no child for which $v.child[i].\sigma \subset \gamma$ (otherwise $S(v.child[i]) \subset v.child[i].\sigma \subset \gamma$).

2. γ crosses the simplices of at most ξ children (this follows from Theorem 4).

3. running *PPRangeEmptiness* on the children whose simplexes cross γ return they are all empty.

Condition 1 is checked by computing $[\![Cont[1]]\!], \ldots [\![Cont[r]]\!]$, where $Cont[i] = 1$ if $v.child[i].\sigma \subset \gamma$ (Line 7). Line 8 computes the product that indicates whether $v.child[i].\sigma \not\subset \gamma$, for all $i = 1, \ldots, r$.

Condition 2 is checked by computing $[\![Cross[1]]\!], \ldots, [\![Cross[r]]\!]$, where $Cross[i] = 1$ if $v.child[i].\sigma$ crosses γ. Then, the number of children whose simplices cross γ is compared to ξ in Line 11.

Finally, to check Condition 3 we use copy-and-recurse. More specifically, we build a selection matrix (Line 12) $M_{\xi \times r}(Cross)$ Line 10. The dimensions of M are $\xi \times r$, where r is the number of children and ξ is an upper bound of the children that cross an empty range γ as follows from Theorem 4. After building M, we use it to make a copy of ξ children (Line 13). Next, we verify for each child that it is empty by calling *PPRangeEmptiness* recursively and setting $[\![e_i]\!]$ to indicate whether $child'[i]$ is empty (Line 15). $[\![Empty_3]\!]$ is then set to 1 if all the checked children are empty by taking the product of $e_1, \ldots, e_\xi$ (Line 16). Eventually, we return $Empty_1 \cdot Empty_2 \cdot Empty_3$ which equals to 1 only if all the conditions above are met.

We conclude with the following theorem which is proved in Appendix A.

Algorithm 2: $PPRangeEmptiness_{n,r,\xi,d,\Gamma}([\![T]\!],[\![\gamma]\!])$

Parameters: n, r, ξ, d, Γ.
Input: (1) A full r-ary (partition) tree T with n leaves, built for d-dimensional points and a range family Γ using a partition theorem (see Section 2.3);
(2) an encrypted segment $[\![\gamma]\!]$, where $\gamma \in \Gamma$.
Output: $[\![Empty]\!]$, where $Empty = \begin{cases} 1 & \text{if } P \cap \gamma = \emptyset \\ 0 & \text{otherwise} \end{cases}$.

Denote by v the root of T.
if *v is a leaf* **then**
  `// check whether` $v.\sigma \subset \gamma$
  $[\![Empty]\!] := 1 - IsContaining([\![v.\sigma]\!],[\![\gamma]\!])$
  Output $[\![Empty]\!]$
else
  foreach $i = 1, \ldots, r$ **do**
    `// check whether` $v.child[i].\sigma \subset \gamma$
    $[\![Cont[i]]\!] := IsContaining([\![v.child[i].\sigma]\!],[\![\gamma]\!])$
  $[\![Empty_1]\!] := \prod(1 - Cont[i])$
  foreach $i = 1, \ldots, r$ **do**
    `// check whether` $c.child[i].\sigma$ `crosses` γ
    $[\![Cross[i]]\!] := IsCrossing([\![v.child[i].\sigma]\!],[\![\gamma]\!])$
  $[\![Empty_2]\!] := isGreater(\sum Cross[i], \xi)$
  $[\![M]\!] := BuildSelectionMatrix_{r,\xi}([\![Cross]\!])$ `// See Section 2.7`
  $[\![child']\!] := M \cdot [\![v.child]\!]$
  foreach $i = 1, \ldots, \xi$ **do**
    $[\![e_i]\!] = PPRangeEmptiness_{\frac{n}{r},r,d,\Gamma}([\![child'[i]]\!],[\![\gamma]\!])$
  $[\![Empty_3]\!] := \prod[\![e_i]\!]$
  Output $[\![Empty_1]\!] \cdot [\![Empty_2]\!] \cdot [\![Empty_3]\!]$

Theorem 6. *Let $P \subset \mathbb{R}^d$ be a set of n points and $\Gamma \subset 2^{\mathbb{R}^d}$, such that for any set $S \subseteq P$ there is a partition Π such that the crossing number of any $\gamma \in \Gamma$ and $P \cap \gamma = \emptyset$ is $\xi = O(r^c)$ then testing whether $P \cap \gamma = \emptyset$ can be answered with a circuit of size $O(t \cdot n^{c+\varepsilon} + n)$ and depth $O(s \cdot \log n)$, where t, s are as in Section 2.6 and $\gamma \in \Gamma$.*

We note there is another case where $P \subset \mathbb{R}^d$ and the ranges are axis-parallel hyper-boxes. This case is a trivial corollary from the improved counting algorithm described in Section 6. In this case counting (and emptiness) can be answered with a circuit whose size is $O(t \cdot \log^d n + n \cdot \log^{d-1})$.

6 Leveled Copy and Recurse

In this section we describe a variation of copy-and-recurse that we call leveled copy-and-recurse. This variation is more efficient in some cases. Specifically it improves the case of axis-parallel hyperboxes mentioned in [KMS24]. Also, it improves the usage of copy-and-recurse to implement traversal-based algorithms such as binary search, B-tree searches [BM70], etc.

In leveled copy-and-recurse we are given a tree T and a plaintext algorithm, $\mathcal{A}$, that traverses it and there is a bound ξ (for some constant $\xi \in \mathbb{N}$) on the number of children of each **node** $\mathcal{A}$ visits in each **level** of T. This is different from the copy-and-recurse described in [KMS24] where for each inner node $\mathcal{A}$ visits there is a bound $O(r^c)$ (for some constant $0 < c < 1$) on the number of its children that $\mathcal{A}$ recurses into. To emphasize, at each level ℓ of T, $\mathcal{A}$ reaches at most ξ nodes and continues into at most ξ children in the next level, $\ell + 1$. This is regardless to how these children relate to the nodes in level ℓ.

Implementing $\mathcal{A}$ with leveled copy-and-recurse works as we now describe. It starts at level $\ell = 1$ of T (i.e. at the root). At each level ℓ, the algorithm acts as follows:

- There are ξ nodes of T that $\mathcal{A}$ visits with a total of $r \cdot \xi$ potential children to continue at. Consider these $r \cdot \xi$ potential nodes and generate an indicator vector for the ξ children (in level $\ell + 1$) that $\mathcal{A}$ actually recurse into.
- Compute the selection matrix M that selects the ξ children in level $\ell + 1$ from the $r \cdot \xi$ potential children.
- Use M to create a copy of the ξ nodes in level $\ell + 1$ that $\mathcal{A}$ traverses into.
- Recurse into the copies of the ξ children.

See for example Algorithm 3 that compute $|P \cap \gamma|$ when $P \subset \mathbb{R}$ and γ is a segment.

Algorithm 3: $PPRangeSearch1D_{n,r}(\llbracket T \rrbracket, \llbracket \gamma \rrbracket)$

Parameters: $n = |P|, r$ (assume n is a power of r)
Input: A full tree T, where v is its root; an encrypted segment $\llbracket \gamma \rrbracket$
Output: $\llbracket x \rrbracket$, where $x = |P \cap \gamma|$.

$\xi := 2$.
$\llbracket x \rrbracket := \llbracket 0 \rrbracket$.
$\llbracket Nodes_1 \rrbracket := (v, \emptyset, \ldots, \emptyset)$
foreach $\ell = 2, \ldots, \log_r n$ **do**
  foreach $i = 1, \ldots, r \cdot \xi$ **do**
    $\llbracket Child_\ell[i] \rrbracket := \llbracket Nodes_{\ell-1}[\lfloor i/r \rfloor].child[i \mod r] \rrbracket$
    $\llbracket Cont_\ell[i] \rrbracket := IsContaining(\llbracket Child_\ell[i].\sigma \rrbracket, \llbracket \gamma \rrbracket)$
    $\llbracket Cross_\ell[i] \rrbracket := IsCrossing(\llbracket Child_\ell[i].\sigma \rrbracket, \llbracket \gamma \rrbracket)$
    $\llbracket x \rrbracket := \llbracket x \rrbracket + \llbracket Cont[i] \rrbracket \cdot \llbracket Child_\ell[i].val \rrbracket$
  $\llbracket M_\ell \rrbracket := BuildSelectionMatrix_{r \cdot \xi, \xi}(\llbracket Cross_\ell \rrbracket)$
  $\llbracket Nodes_\ell \rrbracket := \llbracket M_\ell \rrbracket \cdot \llbracket Child_\ell \rrbracket$
Output $\llbracket x \rrbracket$

Algorithm Overview Algorithm 3 implements range counting for the 1-dimensional case using leveled copy-and-recurse. It gets parameters n and r where $n = |P|$, r is the number of children each inner node has. We also have $\xi = 2$ as follows from Lemma 1. The algorithm gets as input a tree T and a segment γ.

The algorithm traverses the tree from root to leaves. As it does so, for each level ℓ it keeps a vector of ξ nodes, $Nodes_\ell$, that holds the nodes that the plaintext $\mathcal{A}$ would have visited at level ℓ. In the beginning (Line 3) it is set to contain only the root, v (the other elements in $Nodes_1$ are set to $\emptyset$). Then at layer ℓ (for $\ell = 2, \ldots, \log_r n$) Algorithm 3 copies the $r \cdot \xi$ children of the ξ nodes in $Nodes_{\ell-1}$ into the vector $Child_\ell$ (Line 6) and checks for each child $Child[i]$ whether $Child[i].\sigma \subset \gamma$ (Line 7) and whether it crosses γ (Line 8). We add the number of leaves in the subtree of every child that is contained in γ (Line 9). Then, to continue to the next level the algorithm computes the selection matrix, M, that selects the children whose bounding simplex cross γ (Line 10). From Lemma 1 it follows that there are at most $\xi = 2$ children that cross γ because the union of the partitions at level ℓ is itself a partition of P. Finally, the nodes that the plaintext $\mathcal{A}$ visits in level ℓ are set by multiplying $Nodes_\ell := M \cdot Child_\ell$ (Line 11).

6.1 Complexity Analysis

To analyze the size and depth of the circuit of Algorithm 3 we analyze the contribution of each of its elements:

1. Creating the $Child_\ell$ vector (Line 6) requires a circuit of size $O(r \cdot \xi \cdot r^{\log_r n - \ell})$ at level ℓ. For the entire tree that is a total of $\sum O(r \cdot \xi \cdot r^{\log_r n - \ell}) = O(n)$.

2. Checking whether $Child[i].\sigma \subset \gamma$ (Line 7) or whether it crosses γ (Line 8) can be done with a circuit of size $O(t)$ and depth $O(s)$ (see Section 2.6). For the entire tree the size totals to $O(t \cdot \log n)$.

3. Building a selection matrix M_ℓ can be done with a circuit of size $O(\xi^3 \cdot r^2)$ and depth $O(\xi^2 \cdot \log r)$ (see Section 2.7). For the entire tree the size totals to $O(\log n)$.

4. The product $M_\ell \cdot Child_\ell$ can be computed by a circuit of size $O(\xi^3 \cdot r^2)$. For the entire tree that totals to $O(\log n)$.

We conclude with the following lemma.

Lemma 4. *Algorithm 3 can be computed with a circuit of size $O(t \cdot \log n)$ and depth $O(s \cdot \log n)$, where t and s are as in Section 2.6.*

7 Experiments

We now describe the experiments we made to test our algorithms.

What we tested The test case was the 1-dimensional range emptiness problem in which a set $P \subset \mathbb{R}$ of n numbers is preprocessed such that given a query $\gamma = [a, b]$ we can quickly answer whether $P \cap \gamma = \emptyset$. This problem is especially interesting because it is used to search whether a value $p \in \mathbb{N}$ appears in a database.

As mentioned earlier, the emptiness problem can be solved by the closely related counting problem computing $\mu = |P \cap \gamma|$ so our experiments compared 4 algorithms:

1. **Ours.** We implemented Algorithm 2 using leveled copy-and-recurse. Following Lemma 2 we have $\xi = 1$. We tested our implementation with $r = 3, 5, 7, 9$.

2. **Range Counting.** We implemented the algorithm of [KMS24] for range counting. For fairness, we implemented it with leveled copy-and-recurse where we set $\xi = 2$ (following Lemma 1) and $r = 3, 5, 7, 9$. Since the definition of the emptiness problem requires the output to be an emptiness indication (i.e., the value of μ should not leak) after computing μ we output $isEqual(\mu, 0)$, where $isEqual(x, y)$ returns a ciphertext whose message is 1 if $x = y$ and 0 otherwise. The details of $isEqual$ are given below.

3. **Naïve emptiness.** We implemented a naïve algorithm that computes $e_i = isContained(p_i, \gamma)$ for each $p_i \in P$ and then output $1 - \prod(1 - e_i)$.

4. **Naïve counting.** We used the naive counting algorithm that computes $\mu = \sum_i isContained(p_i, \gamma)$. Here also we output $isEqual(\mu, 0)$

What the data was In our experiments the data was a set $P \subset \mathbb{R}$, where $|P| = n$, for $n = 2^{15}, \ldots, 2^{25}$. Each $p_i \in P$ was drawn independently and uniformly from the set $\{0, \frac{1}{100}, \frac{2}{100}, \ldots, 1\}$. For the range γ we drew a uniformly from $\{\frac{1}{200}, \frac{3}{200}, \ldots, \frac{201}{200}\}$ and set $\gamma = [-0.5, a]$.

Our algorithms are generic and can be implemented with any encryption scheme but for our experiments we implemented our algorithms with CKKS where each of $\gamma_a, \gamma_b, p_1, \ldots, p_n$ was encoded as a single message. The key had 18 bits of integer, 42 bits in its fractional part, 12 limbs (multiplication depth) and the security parameter was 128. With these parameters, the key supported bootstrapping and each ciphertext had 2^{15} slots.

Packing and SIMD To utilize all the slots, we partition P into disjoint subsets $P_1, \ldots, P_{2^{15}}$ whose sizes are $|P_i| = n/2^{15}$. We construct a tree T_i for each P_i, where all trees have the same shape. We then construct a tree T of the same shape where each node is a ciphertext and map each node of T_i to the i-th slot of the respective node in T. When answering a range counting query, we compute $\mu_i = |P_i \cap \gamma|$ in parallel for all $i = 1, \ldots, 2^{15}$ trees using SIMD, then using 15 rotations and additions compute $\mu = \sum \mu_i$. Similarly, to answer range emptiness, we compute the indicator χ_i indicating whether $P_i \cap \gamma = \emptyset$ and then using 15 rotations and multiplications compute $\prod_i \chi_i$.

Implementing *isContain* We implemented $isContain(p, \gamma)$ using the primitive $isSmaller(x, y) = (sign(x - y) + 1)/2$ and then setting $isContain(p, \gamma) = isSmaller(\gamma_a, p) \cdot isSmaller(p, \gamma_b)$, where *isSmaller* was implemented using Algorithm 3 from [CKK20]. To remind, given $z = x - y \in [-1, 1]$ they approximated $sign(z) = f_d^{(d_f)} \circ g_d^{(d_g)}(z)$, where the degree of the f_d and g_d polynomials is $2d + 1$, also d_f and d_g is the number of times f and g are composed. In our implementation the degree of f and g was 7 and we fixed $d_f = 1$ and $d_g = 4$.

The setup Our algorithms are generic and can be implemented with any scheme that supports additions and multiplications. For our experiments we used the CKKS scheme [CKKS17] which is a popular scheme. In our implementation we used the HELayers framework [AAB+20] which is built on top of many cryptographic libraries. For our experiments we used the CKKS implementation in the HEaaN library [Cry22]. The system we used had 32 cores (128 threads) of AMD EPYC 7742 CPU with 500 GB memory and NVIDIA A100-SXM4-80GB GPU.

The results The results are summarized in Table 2 and in the graphs below. Table 2 compares the time to run an emptiness query in the 1 dimensional case, i.e., $P \subset \mathbb{R}$ and $\gamma = [a, b] \subset \mathbb{R}$ is a segment. The sizes of the set P were comparable to sizes of real data sets: $|P| = 2^{15}, \ldots, 2^{25}$. The first 4 rows show the time of our algorithm (Item 1 above) The 5^{th} row shows the time to run the naïve emptiness algorithm (Item 3 above). The next 4 rows show the time to run the algorithm of [KMS24] (Item 2 above). The last row shows the time of the naïve counting (Item 4 above).

For example, for a data set with $|P| = 2^{23}$, our algorithm (with $r = 3$) takes 28 seconds whereas running the counting algorithm of [KMS24] (with $r = 3$) takes 93.3 seconds. Running the naïve emptiness takes 192.8 seconds and running the naïve counting takes 177.6 seconds. Our algorithm was faster than that of [KMS24] for all choices of r and all sizes of P that we checked. This is consistent with our analysis since our emptiness algorithm was implemented with leveled copy-and-recurse with $\xi = 1$ (compared to $\xi = 2$ for [KMS24]).

Our algorithm is faster than the naïve counting algorithm for data sets larger than $|P| > 2^{21}$, whereas the algorithm of [KMS24] is faster than the naïve counting algorithm only when $|P| > 2^{23}$. The naïve emptiness algorithm is slower than the naïve counting algorithm for all data sets that we checked and is probably because it requires more multiplications.

The results are also summarized in Figure 5 and Figure 6. The first figure compares our algorithm with $r = 3$ to the naïve algorithm that is based on the naïve counting algorithm. The x-axis is log-scaled and it shows the data set size $|P|$. The y-axis is also log-scaled and shows the time in seconds

Table 2: A summary of the time to compute emptiness queries for a set $P \subset \mathbb{R}$. All times are given in seconds. The first 4 rows show the time for our algorithm with $r = 3, 5, 7, 9$. The 5^{th} row shows the time for the naïve emptiness algorithm. The next 4 rows show the time for the counting algorithm of [KMS24] with $r = 3, 5, 7, 9$. The last row shows the time for the naïve counting algorithm.

$\lvert P\rvert$	2^{15}	2^{17}	2^{19}	2^{21}	2^{23}	2^{25}
Ours (r=3)	1.3	4.6	14.4	27.3	28.0	68.9
Ours (r=5)	1.3	3.6	17.0	33.9	39.5	104.9
Ours (r=7)	1.3	3.6	16.7	29.8	81.1	184.7
Ours (r=9)	1.3	3.7	16.1	69.0	69.9	150.7
Naïve Emptiness	1.4	3.7	12.7	48.9	192.8	771.8
[KMS24] (r=3)	1.4	8.0	32.3	87.4	93.3	248.7
[KMS24] (r=5)	1.4	5.6	32.5	88.4	134.8	323.5
[KMS24] (r=7)	1.4	5.6	31.6	74.6	239.4	538.7
[KMS24] (r=9)	1.4	5.7	30.0	139.7	202.2	427.0
Naïve Counting	0.7	2.8	11.2	44.8	177.6	714.3

to run the algorithms. The naïve algorithm runs in linear time $O(t \cdot n)$ and appears as a straight line (we remind that when the x and y axes are log-scaled any algorithm with running time $O(n^d)$ appears as a straight line whose slope is determined by d). The running time of our algorithm is higher than the naïve for small values of n; they have the same running time for $n \approx 2^{20}$; and our algorithm is faster when $n > 2^{20}$. Furthermore, the slope of our algorithm is smaller than the slope of the naive algorithm, indicating that the time difference between the algorithms gets bigger as n increases. This matches our analysis that the running time of our algorithm is $O(t \cdot \log n + n)$.

Figure 6 compares our algorithm with different values or r. As our experiments show, the case of $r = 3$ is the fastest while $r = 7$ is slowest (when $n = 2^{23}, 2^{25}$).

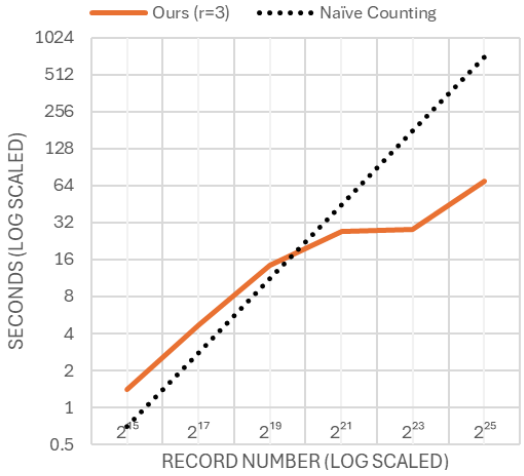

Figure 5: Running times of Algorithm 2 ($r = 3$) and of the naïve counting algorithm.

8 Conclusion and future work

In this paper we showed 3 improvements to the results in [KMS24].

First, we introduced a leveled version of the copy-and-recurse technique for traversing trees with a bound ξ on the number of nodes visited per level. This approach enables efficient 1-dimensional range queries with a circuit size of $O(t \cdot \log n + n)$. Leveled copy-and-recurse can also be applied to search full binary trees and B-trees, suggesting broader applications in future work.

Then, we optimized the overhead term in the range-searching algorithm from [KMS24], reducing it from $O(n^{1+\epsilon})$ to the optimal $O(n)$. This is achieved through a tighter formulation of the partition theorems in [Mat92, Mat91, AM94, SS11]. For instance, with the improved partition theorem d-dimensional range queries are answered in $O(t \cdot n^{1-1/c+\epsilon} + n)$ time, where $c = d$ for $d \leq 4$ and $c = 2d - 4$ for $d > 4$.

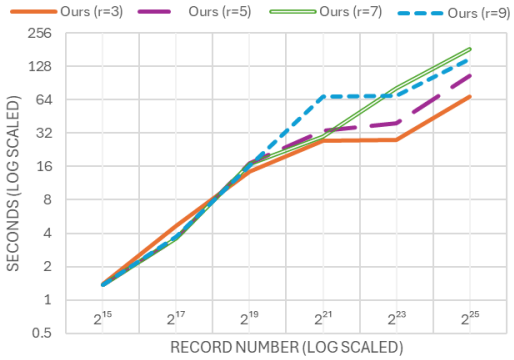

Figure 6: Running times of Algorithm 2 with $r = 3, 5, 7, 9$.

Finally, we demonstrated how to answer range emptiness queries more efficiently than range counting by leveraging the partition theorems from [Mat92, SS11]. Emptiness queries are valuable on their own and also serve as fundamental components in other algorithms, such as the approximate counting method in [AHP08].

Finally, we implemented our FHE-based emptiness query approach and confirmed that it is indeed faster than range counting.

References

[AAB+20] Ehud Aharoni, Allon Adir, Moran Baruch, Nir Drucker, Gilad Ezov, Ariel Farkash, Lev Greenberg, Ramy Masalha, Guy Moshkowich, Dov Murik, Hayim Shaul, and Omri Soceanu. HeLayers: A Tile Tensors Framework for Large Neural Networks on Encrypted Data. *CoRR*, abs/2011.0, 2020.

[ADE+23] Ehud Aharoni, Nir Drucker, Gilad Ezov, Eyal Kushnir, Hayim Shaul, and Omri Soceanu. Poster: Efficient aes-gcm decryption under homomorphic encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS '23, page 3567–3569, New York, NY, USA, 2023. Association for Computing Machinery.

[AHP08] Boris Aronov and Sariel Har-Peled. On approximating the depth and related problems. *SIAM Journal on Computing*, 38(3):899–921, 2008.

[AM94] Pankaj K. Agarwal and Jiří Matoušek. On range searching with semialgebraic sets. *DISCRETE COMPUT. GEOM*, 11:393–418, 1994.

[AMS13] Pankaj K. Agarwal, Jiří Matoušek, and Micha Sharir. On range searching with semialgebraic sets. ii. *SIAM Journal on Computing*, 42(6):2039–2062, 2013.

[Bea92] Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *Advances in Cryptology — CRYPTO '91*, pages 420–432, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.

[BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS '12, pages 309–325, New York, NY, USA, 2012. ACM.

[BIM00] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers computation in private information retrieval: Pir with preprocessing. In *Annual International Cryptology Conference*, pages 55–73. Springer, 2000.

[BM70] R. Bayer and E. McCreight. Organization and maintenance of large ordered indices. In *Proceedings of the 1970 ACM SIGFIDET (Now SIGMOD) Workshop on Data Description, Access and Control*, SIGFIDET '70, page 107–141, New York, NY, USA, 1970. Association for Computing Machinery.

[CFLM20] Sherman S. M. Chow, Katharina Fech, Russell W. F. Lai, and Giulio Malavolta. Multi-client oblivious ram with poly-logarithmic communication. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, pages 160–190, Cham, 2020. Springer International Publishing.

[CGKO06] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*, CCS '06, page 79–88, New York, NY, USA, 2006. Association for Computing Machinery.

[CKK15] Jung Hee Cheon, Miran Kim, and Myungsun Kim. Search-and-compute on encrypted data. In *International Conference on Financial Cryptography and Data Security*, pages 142–159. Springer, 2015.

[CKK20] Jung Hee Cheon, Dongwoo Kim, and Duhyeong Kim. Efficient homomorphic comparison methods with optimal complexity. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 221–256. Springer, 2020.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437, Cham, 2017. Springer International Publishing.

[Cry22] CryptoLab. HEaaN: Homomorphic Encryption for Arithmetic of Approximate Numbers, version 3.1.4, 2022.

[DPP+16] Ioannis Demertzis, Stavros Papadopoulos, Odysseas Papapetrou, Antonios Deligiannakis, and Minos Garofalakis. Practical private range search revisited. In *Proceedings of the 2016 International Conference on Management of Data*, pages 185–198, 2016.

[DPP+18] Ioannis Demertzis, Stavros Papadopoulos, Odysseas Papapetrou, Antonios Deligiannakis, Minos Garofalakis, and Charalampos Papamanthou. Practical private range search in depth. *ACM Transactions on Database Systems (TODS)*, 43(1):1–52, 2018.

[dua24] Duality query engine. https://dualitytech.com/platform/duality-query/, 2024.

[FJK+15] Sky Faber, Stanislaw Jarecki, Hugo Krawczyk, Quan Nguyen, Marcel Rosu, and Michael Steiner. Rich queries on encrypted data: Beyond exact matches. In Günther Pernul, Peter Y A Ryan, and Edgar Weippl, editors, *Computer Security – ESORICS 2015*, pages 123–145, Cham, 2015. Springer International Publishing.

[FMET22] Francesca Falzon, Evangelia Anna Markatou, Zachary Espiritu, and Roberto Tamassia. Range search over encrypted multi-attribute data. *Cryptology ePrint Archive*, 2022.

[GJW19] Zichen Gui, Oliver Johnson, and Bogdan Warinschi. Encrypted databases: New volume attacks against range queries. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, page 361–378, New York, NY, USA, 2019. Association for Computing Machinery.

[GLMP18] Paul Grubbs, Marie-Sarah Lacharite, Brice Minaud, and Kenneth G. Paterson. Pump up the volume: Practical database reconstruction from volume leakage on range queries. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, page 315–331, New York, NY, USA, 2018. Association for Computing Machinery.

[hea24] Heaan homomorphic analytics. https://www.ncloud.com/product/analytics/heaanHomomorphic, 2024.

[ibm24] Ibm z solutions. https://www.ibm.com/support/z-content-solutions/fully-homomorphic-encryption/, 2024.

[iee08] Ieee standard for floating-point arithmetic. Standard IEEE Std 754-2008, IEEE Computer Society, New York, NY, USA, August 2008.

[KMS24] Eyal Kushnir, Guy Moshkowich, and Hayim Shaul. Secure range-searching using copy-and-recurse. *Proceedings on Privacy Enhancing Technologies*, 2024(3):626–644, 2024.

[LMP18] Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. Improved reconstruction attacks on encrypted data using range query leakage. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 297–314, 2018.

[Mat91] Jiří Matoušek. Efficient partition trees. In *Proceedings of the Seventh Annual Symposium on Computational Geometry*, SCG '91, page 1–9, New York, NY, USA, 1991. Association for Computing Machinery.

[Mat92] Jiří Matoušek. Reporting points in halfspaces. *Computational Geometry*, 2(3):169–186, 1992.

[MFET23] Evangelia Anna Markatou, Francesca Falzon, Zachary Espiritu, and Roberto Tamassia. Attacks on Encrypted Response-Hiding Range Search Schemes in Multiple Dimensions. *Proceedings on Privacy Enhancing Technologies*, 2023(4):204–223, 2023.

[MMRS15] Matteo Maffei, Giulio Malavolta, Manuel Reinert, and Dominique Schröder. Privacy and access control for outsourced personal records. In *2015 IEEE Symposium on Security and Privacy*, pages 341–358, 2015.

[NM90] Stefan Näher and Kurt Mehlhorn. Leda: a library of efficient data types and algorithms. In *Proceedings of the Seventeenth International Colloquium on Automata, Languages and Programming*, page 1–5, Berlin, Heidelberg, 1990. Springer-Verlag.

[SS11] Micha Sharir and Hayim Shaul. Semialgebraic range reporting and emptiness searching with applications. *SIAM Journal on Computing*, 40(4):1045–1074, 2011.

[TCBS23] Daphné Trama, Pierre-Emmanuel Clet, Aymen Boudguiga, and Renaud Sirdey. A homomorphic aes evaluation in less than 30 seconds by means of tfhe. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC '23, page 79–90, New York, NY, USA, 2023. Association for Computing Machinery.

[TCSK16] Gamze Tillem, Ömer Mert Candan, Erkay Savaş, and Kamer Kaya. Hiding access patterns in range queries using private information retrieval and oram. In Jeremy Clark, Sarah Meiklejohn, Peter Y.A. Ryan, Dan Wallach, Michael Brenner, and Kurt Rohloff, editors, *Financial Cryptography and Data Security*, pages 253–270, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.

[U.S96] U.S. Congress. Health Insurance Portability and Accountability Act of 1996. https://www.hhs.gov/hipaa/, 1996. Public Law 104-191.

A Proof of Theorem 6

We now turn to prove Theorem 6, namely the size and depth of the circuit that realizes Algorithm 2. To do that we first analyze the complexity of each step and then write and solve the recursion equation.

Computing $Empty_1$ involves computing $IsContaining$ for all r children. This is done in a circuit of size $O(r \cdot t)$ and depth $O(s)$.

Computing $Empty_2$ involves computing $IsCrossing$ for all r children. This is done in a circuit of size $O(r \cdot t)$ and depth $O(s)$.

Computing $Empty_3$ involves: (1) building the selection matrix M, which is done in a circuit of size $O(r^2 \cdot \xi)$ and depth $O(\log r)$ (see Section 2.7); (2) multiplying M by the vector of all r children and their subtrees, which is done in a circuit of size $O(r \cdot (n/r) \cdot \xi)$ and depth $O(\log r)$; (3) applying $PPRangeEmptiness$ on the ξ copies children. Computing $Cross[1], \ldots, Cross[r]$ is not mentioned here because it is charged to computing $Empty_2$.

Let n be the number of leaves in the partition tree. The circuit size bound $T(n)$ to compute *PPRangeEmptiness* is therefore

$$T(n) = O(r \cdot t) + O(r \cdot t) + O(r^2 \cdot \xi + n \cdot \xi + \xi \cdot T(n/r)) = O(r \cdot t + n \cdot \xi + \xi T(n/r)).$$

When $\xi = r^c$. This solves to

$$T(n) = O(r \cdot t \cdot n^{c+\varepsilon} + n \cdot r^c) = O(t \cdot n^{c+\varepsilon} + n). \tag{2}$$

We conclude this section by restating Lemma 6:

Theorem 6. *Let $P \subset \mathbb{R}^d$ be a set of n points and $\Gamma \subset 2^{\mathbb{R}^d}$, such that for any set $S \subseteq P$ there is a partition Π such that the crossing number of any $\gamma \in \Gamma$ and $P \cap \gamma = \emptyset$ is $\xi = O(r^c)$ then testing whether $P \cap \gamma = \emptyset$ can be answered with a circuit of size $O(t \cdot n^{c+\varepsilon} + n)$ and depth $O(s \cdot \log n)$, where t, s are as in Section 2.6 and $\gamma \in \Gamma$.*

B Modifying Other Partition Theorems

In Section 4 we showed a tighter version of the partition theorem of [Mat91]. Here we list other partition theorems (specifically, those in [AM94, Mat92, SS11]) that can be improved in a similar way. The proofs of these theorems are different from that of [Mat91] in how they construct the weighted cutting or the a test-set Q but they follow the same outline. We state the modified versions of Theorem 3.1. in [Mat92] and then of Theorem 3.2 in [SS11] without proving them.

Theorem 7. *Let $P \in \mathbb{R}^d$ be a set of n points, let $k, r \in \mathbb{N}$ be parameters, where r divides n and $k < n/r$. Then there exists a simplicial partition $\Pi = ((P_1, \sigma_1), \ldots, (P_r, \sigma_r))$, that satisfies $|P_i| = n/r$ such that the crossing number of any k-shallow hyperplane relative to Π is $O(r^{1-1/\lfloor d/2 \rfloor})$.*

Theorem 8. *Let P be a set of n points in $\mathbb{R}^d$, let Γ be a family of semi-algebraic ranges of constant description complexity, and let r be fixed. Let Q be another finite collection (not necessarily a subset of Γ) of semi-algebraic ranges of constant description complexity with the following properties: (i) The ranges in Q are all (n/r)-shallow. (ii) The complement of the union of any m ranges of Q can be decomposed into at most $\zeta(m)$ elementary cells. (iii) Any (n/r)-shallow range $\gamma \in \Gamma$ can be covered by the union of $O(1)$ ranges of Q. Then there exists an elementary cell partition $\Pi = ((P_1, \sigma_1), \ldots, (P_r, \sigma_r))$, where $|P_i| = n/r$, such that the crossing number of any $(n/r) - $ shallow range in Γ with the cells of Π is $O(r/\zeta^{-1}(r) + \log r \log |Q|)$, if $\zeta(r) = \Omega(r^{1+\varepsilon})$, for any fixed $\varepsilon > 0$, or $O(r \log r/\zeta^{-1}(r) + \log r \log |Q|)$, otherwise.*

We plug bounds on ξ (from Theorem 4) into Equation 2 to get the corollary below.

Given a set $P \subset \mathbb{R}^d$ of n points, we can construct an encrypted data structure for a family of ranges Γ that given an encrypted range $\gamma \in \Gamma$ can answer emptiness queries in a circuit of size $T(n)$ and depth $O(s \cdot \log n)$, where:

1. when $P \subset \mathbb{R}^d$ is a set of points Γ is the family of all halfspaces bounded by a hyperplane, then we can answer half-space emptiness in a circuit of size $T(n) = O(t \cdot n^{1-1/\lfloor d/2 \rfloor + \varepsilon} + n)$.
2. When P is a set of n balls in 3-space and $\Gamma = \{$segments in 3-space$\}$, then we can answer whether a segment does not intersect any ball in a circuit of size $T(n) = O(t \cdot n^{2/3+\varepsilon} + n)$.
3. When $P \subset \mathbb{R}^3$ is a set of points and $\Gamma = \{$convex objects$\}$, then we can answer whether $\gamma \in \Gamma$ contains all the points in P (i.e., the complement to γ is empty) in a circuit of size $T(n) = O(t \cdot n^{1/2+\varepsilon} + n)$.
4. When $P \subset \mathbb{R}^2$ is a set of points and $\Gamma = \{\alpha$-fat trianlge$\}$ or $\Gamma = \{$large circular cap$\}$, then we can answer whether γ is empty in a circuit of size $T(n) = O(t \cdot n^{\varepsilon} + n)$.

Proof. 1. For the case of hyperplanes-bounded halfspace ranges amid points in $\mathbb{R}^d$, Matoušek [Mat92] showed a partition that leads to answering a query in $O(n^{1-1/\lfloor d/2 \rfloor + \varepsilon})$ time. Using the bound from Theorem 7 and the copy-and-recurse technique of [KMS24] we get that testing whether a hyperplane-bounded halfspace is empty can be tested with a circuit of size $O(t \cdot n^{1-1/\lfloor d/2 \rfloor + \varepsilon} + n)$.

2. For the case of 3D-segment ranges amid spheres in $\mathbb{R}^3$, [SS11] showed a plaintext solution that answers a query in $O(n^{2/3+\varepsilon})$ time. Using the bound from Theorem 8 and the copy-and-recurse technique of [KMS24] we get that testing whether a segment does not intersect any sphere can be done with a circuit of size $O(t \cdot n^{2/3+\varepsilon} + n)$.

3. For the case of a range family of convex shapes amid points in $\mathbb{R}^3$, [SS11] showed a plaintext solution that answers a query in $O(n^{1/2+\varepsilon})$ time. Using the bound from Theorem 8 and the copy-and-recurse technique of [KMS24] we get that testing whether a range contains all the points of P can be answered using a circuit of size $O(t \cdot n^{2/3+\varepsilon} + n)$.

4. For the case of a range family of α-fat triangles or a family of large circular caps amid points in $\mathbb{R}^2$, [SS11] showed a plaintext solution that answers a query in $O(n^{\varepsilon})$ time. Using the bound form Theorem 8 and the copy-and-recurse technique of [KMS24] we get that testing whether a α-fat triangle range or a circular-cap range is empty can be answered with a circuit of size $O(t \cdot n^{\varepsilon} + n)$.

□

We note that the proof of the partition theorem in [AMS13] follows a different outline - one that is based on algebraic geometry. In this case, it is not clear whether their partition theorem can be modified in a similar way. We leave this for future work.

C Approximate Counting Under FHE

Given a set P and $\delta > 0$, the problem of approximate counting is to preprocess P such that given γ we can efficiently compute μ_γ such that

$$(1-\delta)|P \cap \gamma| \leq \mu_\gamma \leq (1+\delta)|P \cap \gamma|.$$

Specifically, note that when $P \cap \gamma = \emptyset$ then we have exactly $\mu_\gamma = |P \cap \gamma|$. Motivated by the example in Section 1 to compute conditional probabilities we can use a multiplicative approximation to estimate

$$\frac{|a \cap b|(1+\delta_1)}{|b|(1+\delta_2)} = \frac{\Pr[a \cap b](1+\delta_1)}{\Pr[b](1+\delta_2)} = \frac{\Pr[a \cap b]}{\Pr[b]}(1+\delta_3) \approx \Pr[a \mid b],$$

where $\Pr[a \cap b](1+\delta_1)$ and $\Pr[b](1+\delta_2)$ can be computed by approximately counting the elements in the respective ranges.

We note that approximating the count with an additive factor (i.e., $|P \cap \gamma| - 1/\delta \leq \mu_\gamma^+ \leq |P \cap \gamma| + 1/\delta$) is easily done by choosing a random subset $S \subset P$ where $|S| = |P| \cdot \delta$ and then setting $\mu_\gamma^+ = |S \cap \gamma|/\delta$, where $0 < \delta < 1$ is a parameter that trades the accuracy of the approximation with running time. Unfortunately, when $\Pr[a \cap b]$ or $\Pr[b]$ are small the approximation does not hold. For example, $\Pr[a \mid b] \not\approx \frac{\Pr[a \cap b]+1/\delta_1}{\Pr[b]+1/\delta_2}$, when $\Pr[a \cap b] \approx 1/\delta_1$.

In [AHP08] the authors showed how to use emptiness range searching to compute an approximate count. Their idea was to generate multiple independent samples $S_1, S_2 \ldots \subset P$ of various sizes and then estimate $|P \cap \gamma|$ by counting the number of samples for which $S_i \cap \gamma = \emptyset$. We show how to implement the work of [AHP08] under FHE

C.1 Emptiness to Approximate Counting (in Plaintext)

We start by discussing the work of [AHP08] to solve approximate counting in plaintext using emptiness queries. In a nutshell, their idea is to use multiple samples of various sizes and test γ on these samples. By observing the number of samples in which γ was empty we can approximate $|\gamma \cap P|$. We next give the details needed to understand our result.

From Emptiness to Approximate Counting The authors of [AHP08] showed how to use data structures that test emptiness queries to efficiently find the approximation $(1-\delta)P_\gamma < \mu_\gamma < (1+\delta)P_\gamma$. In what follows we review their solution. We try to avoid the gory details and report only the details that are relevant to our FHE implementation.

The main idea behind the work of [AHP08] is to construct independent random samples of P with various sizes. Testing the emptiness on these samples can tell us (with high probability) the approximation μ_γ. Intuitively, if P_γ is small then many samples would be empty, while if P_γ is large then many samples would be non-empty.

Database to compare P_γ to a parameter z. They start with constructing a data structure $\mathcal{D}(z,\delta)$ that determines whether $(1-\delta)\mu_\gamma > z$ or $(1+\delta)\mu_\gamma < z$. Specifically, when $z \in [\mu(1-\delta), \mu(1+\delta)]$ then $\mathcal{D}(z,\delta)$ may return either outputs.

To construct $\mathcal{D}(z,\delta)$ they sample $M = M(\delta) = \lceil c_3\delta^{-2}\log n\rceil$ subsets $\mathcal{R}_1,\ldots,\mathcal{R}_M \subseteq P$ by picking every element $p \in P$ with probability $1/z$, where c_3 is a sufficiently large constant. A data structure D_i that answers emptiness queries is built for each R_i. To decide whether $P_\gamma < z$ with a margin of error δ, they define a Bernoulli random variable $X_i = \begin{cases} 1, & \text{if } \gamma \cap R_i \neq \emptyset \\ 0, & \text{otherwise} \end{cases}$, for $i = 1,\ldots,M$. The value of X_i is determined using D_i. For a range γ with $P_\gamma = k$, we have $Pr[X_i = 1] = \rho(k) = 1 - (1-\frac{1}{z})^k$. Set $Y = \sum_{1\le i\le N} X_i$. Now, if $P_\gamma = z$ then $E[Y] = N\cdot\rho(z) = \Delta$. After computing Y, $\mathcal{D}(z,\delta)$ returns "$P_\gamma < z$" (LOW) if $Y < \Delta$ and "$P_\gamma \ge z$" ($HIGH$) otherwise.

They summarize this data structure in the following Lemma.

Lemma 5 (Lemma 5.2 and Lemma 5.3 from [AHP08])**.** *Given a set P of n objects, a parameter $0 < \delta$, and $z \in [0,n]$, one can construct a data structure $\mathcal{D}(z)$ which given a range γ returns either LOW or HIGH. If it returns LOW, then $\mu_\gamma \le (1+\delta)z$, and if it returns HIGH then $\mu_\gamma \ge (1-\delta)z$. The data structure might return either answer if $\mu_\gamma \in [(1-\delta)z, (1+\delta)z)]$.*

The data structure $\mathcal{D}$ consists of $M = O(\delta^{-2}\log n)$ emptiness data structures. The space and time needed for $\mathcal{D}$ are $O(S(2n/z)\delta^{-2}\log n)$ and $O(Q(2n/z)\delta^{-2}\log n)$ (respectively), where $S(m)$ and $Q(m)$ are the space and time needed for a single emptiness data structure storing m objects, respectively. All bounds hold with high probability.

An easy way to find an approximation μ_γ is to use Lemma 5 to construct $\mathcal{D}_1,\ldots,\mathcal{D}_W$, where $W = O(\delta^{-1}\log n)$ and carefully select values $z_1,\ldots,z_W$. In [AHP08] they set

$$z_w = \begin{cases} w/2, & \text{when } w < U \\ (U/4)(1+\delta/16)^w, & \text{otherwise} \end{cases},$$

where $U = O(1/\delta)$. Then, $\mathcal{D}_1,\ldots,\mathcal{D}_W$ are used to search for μ_γ. For convenience, we summarize this in the following lemma.

Theorem 9 (Theorem 5.6 from [AHP08])**.** *Given a set P of n points and a data structure that answers emptiness queries in $Q(n)$ time using $S(n)$ space, one can construct a data structure that uses $O(\delta^{-2}S(n)\log n)$ space and that, given a range γ, outputs a number μ_γ with $(1-\delta)P_\gamma \le \mu_\gamma \le P_\gamma(1+\delta)$, where $P_\gamma = |P\cap\gamma|$. The query time is $O(\delta^2 Q(n)\log n)$. The output of the query is correct with high probability for all queries and the running time bounds hold with high probability*

C.2 Approximate Counting Under FHE

We now show how to implement the work of [AHP08] under FHE. Like the authors of [AHP08] we construct the data structures $\mathcal{D}_1,\ldots,\mathcal{D}_W$, however the (clever) way they search over them does not extend well to FHE because it involves many branching. Instead we test $\mathcal{D}_w$ for all w and return n_w associated with the first $\mathcal{D}_w$ that returned "HIGH". The question whether a more efficient search can be done under FHE is left for future research. Our algorithm is described below.

Algorithm Overview Algorithm 4 has 3 parameters: $n = |P|$; δ - the required approximation accuracy and $W = O(\delta^{-1}\log n)$. The algorithm also receives as input a range $\gamma \in \Gamma$ and data structures $\mathcal{D}_1,\ldots,\mathcal{D}_W$ that were constructed as desctibed in [AHP08]. To iterate, for each w we have $\mathcal{D}_w = (D_1^w,\ldots,D_M^w)$, where $M = O(\log n)$ and each D_i^w is a data structure built to answer emptiness queries over a random sample of approximately n_w points (each point $p \in P$ is chosen with probability $\frac{n_w}{n}$). As shown in [AHP08], Y'_w is some constant that depends on the parameters and it is determined that $\mu < n_w$ if $Y'_w < Y_w$, where Y'_w is the number of data structures from $D_1^w,\ldots,D_M^w$ that reported that γ is empty. The algorithm therefore computes Y'_w (Line 2) and then compares Y'_w to Y_w to determine whether the output of $\mathcal{D}_w$ is that $\mu < n_w$ (Line 3).

Algorithm 4: $ApproxCounting_{n,W,\delta}([\![\mathcal{D}_1]\!],\ldots,[\![\mathcal{D}_W]\!],[\![\gamma]\!])$

Parameters: $n=|P|, W, \delta$
Input: Data structures $\mathcal{D}_1,\ldots,\mathcal{D}_W$,
where each $\mathcal{D}_w=(D_1^w,\ldots,D_M^w)$ and D_i^w is a data structure
for emptiness queries for a sample of n_w points;
an encrypted range $[\![\gamma]\!]$
Output: $[\![x]\!]$, where $|P\cap\gamma|(1-\delta)\le x\le|P\cap\gamma|(1+\delta)$.

1 **foreach** $w=1,\ldots,W$ **do**
2 $\quad Y'_w:=\sum_{i=1}^{M} PPRangeEmptiness(D_i^w,\gamma)$
3 $\quad H_w:=isSmaller(Y'_w,Y_w)$
4 **Output** $n_1\cdot H_1+\sum_{w=2}^{W} n_w\cdot(H_w-H_{w-1})$

The output of the algorithm is the same as that of [AHP08], namely, n_w, where w is the lowest such that $H_w=1$, i.e., the first $\mathcal{D}_w$ that output $\mu>n_w$. That is calculated in Line 4 as we now explain. The values of $\{H_w\}$ are $\ldots,0,0,1,1,\ldots$, where $H_w=0$ if $\mathcal{D}_w$ returned "L" and $H_w=1$ otherwise. It easily follows that $(H_w-H_{w-1})=1$ only when $H_w=1$ and H_{w-1} canceling all but the correct output n_w which is a good approximation for $|P\cap\gamma|$, as shown by [AHP08].

Circuit-Size Analysis The size of the circuit that realizes Algorithm 4 is easily given by:

$$\begin{aligned}T(n)=\sum_w T_{\mathcal{D}_w}=\sum_w T_{D^w}(n_w)\cdot M=O(\log n)\cdot\sum_w T_{D^w}(n_w)\\ =O(\log n)(\sum_{w=1}^{U} T_{D^w}+\sum_{w=U+1}^{W} T_{D^w})\\ =O(\log n)\left(\sum_{w=1}^{U} Q\Big(\frac{w}{2}\Big)+\sum_{w=U+1}^{W} Q\Big(\frac{U}{4}(1+\varepsilon/16)^w\Big)\right),\end{aligned}$$

where $T_{\mathcal{D}_w}$ is the circuit size that checks the $\mathcal{D}_w$ data structure, T_{D_w} is the circuit size that checks the D^w data structure and $Q(n)$ is the circuit size that answers an emptiness query over n points. For example, when $Q(n)=O(t\cdot n^{c+\varepsilon}+n)$ this solves to

$$T(n)=O(t\cdot n^{c+\varepsilon}+n\log n),$$

and when $Q(n)=O(t\cdot\log n+n)$ this solves to

$$T(n)=O(t\log^3 n+n\log n).$$